プロジェクト管理入門

あるいは、ニック・ジェンキンスによる

「プロジェクトを成功させるためのガイド」。

©Nick Jenkins, 2005

この作品はクリエイティブ・コモンズ（表示-非営利-継承）の下でライセンスされています。2.5ライセンス...このライセンスのコピーを閲覧するには、[http://creativecommons.org/licenses/by-nc- sa/2.5/]にアクセスするか、(b) "Creative Commons, 543 Howard Street, 5th Floor,San Francisco, California, 94105, USA "に手紙を

送る。

要約すると、あなたはこの作品を自由に複製、頒布、展示、上演し、二次的著作物を作成することができます。この作品を商業目的で使用することはできません。また、この作品を改変、変形、構築する場合、このライセンスと同一のライセンスの下でのみ、その結果としての作品を頒布することができます。再利用または頒布する場合、あなたはこの作品のライセンス条件を他の人に明らかにしなければなりません。著作権者から許可を得れば、これらの条件のいずれも放棄することができます。あなたの公正使用およびその他の権利は、上記によって何ら影響を受けるものではありません。詳細はライセンスをご覧ください。

目次

はじめに 3

基本原則 4

成功のための10の公理 4
スコープ・トライアングル 6
クリティカル・パス 7
神話の男」月間 8

スコープ 10

範囲、ビジョン、目標 10
まともな提案 12
必要条件 15
要件キャプチャ 17
要件の文書化 19
トレーサビリティ 22

プランニング 23

プロジェクト計画の目的 23
スケジューリングの極意 24
原価計算と予算管理 29
リスク管理 31
チェンジ・マネジメント 33

実行 35

軌道を維持する 35
人材管理 36

実装 **39**

レビュー **42**

用語集 **43**

国際大会

th多くのプロジェクトが失敗するのは、最も単純な原因によるものだ。プロジェクトを予定通りに進めるのに天才である必要はないし、プロジェクトマネジャーになるのに神秘的なプロジェクトマネジメントの方法論に精通している必要もない。もしこれを読んでも、平均的な能力の持ち主がプロジェクトを成功させることができないなら、私は75歳の誕生日にタイムズスクエアを全裸で走り抜けるだろう！

あるジョークを思い出したよ...。

ある観光客がペットショップに入り、陳列されている動物を眺めていた。店員はうなずき、店の脇にある檻に行き、サルを取り出した。首輪とリードをつけ、"5,000ポンドです "と言って客に渡した。

驚いた観光客は店主のところに行き、こう言った。店主は答えた。"ああ、あのサルはC言語でプログラミングができるんだ。

観光客は別の檻の中のサルに目をやった。「あれはもっと高いよ！ £10,000!何をするんだ?

「オブジェクト指向プログラミング、Visual C++、Javaも扱えます。本当に便利なものばかりです」と店員は言った。

首には50,000ポンドの値札がついていた。観光客は店員に「あれは他の猿を全部合わせた値段よりも高いんですよ」と言った！ いったい何をするんだ? "

店主は、「まあ、実際に何かをするのを見たことはありませんが、プロジェクト・マネージャーだと書いてありますよ」と答えた。

成功のための10の公理

あなたが始めるのを助けるために、ここに10の（自明な）真実がある：

I.　目標を知る

当たり前のことのように聞こえますが、最終地点が決まっていなければ、そこに到達することはできません。誰が見てもわかるように、プロジェクトのゴールを明確に述べることができなければなりません。もしあなたのゴールを一文で適切に説明できなければ、それを達成できる可能性はかなり低いでしょう。

II.　チームを知る

チームは最も重要なリソースであり、彼らの熱心な貢献がプロジェクトを左右する。彼らの世話をし、チームが個人の集まりではなく、ひとつのユニットとして機能するようにしましょう。コミュニケーションは不可欠です！ 信頼関係を促進し、自分が大局に何を貢献しなければならないかを全員が理解できるようにするために時間を割きましょう。批判だけでなく報酬も与え、優れた労働条件を提供し、率先垂範すること。

III.　ステークホルダーを知る

利害関係者と時間を過ごす。ステークホルダーは、プロジェクトに専門的な知識を提供してくれるか、成功に不可欠な政治的、商業的な支持を提供してくれるだろう。必要に応じて握手やキスを交わし、官僚の歯車に油を差すことで、プロジェクトが可能な限りスムーズに進むようにする。

IV.　プランニングとデザインに時間をかける

プロジェクトで伝統的に犯されてきた大きな過ちは、準備が整う前に飛躍してしまうことだ。納品しなければならないというプレッシャーにさらされていると、「ボールを転がそう」という誘惑に駆られる。しかし、ボールは大きくて重く、いったん動き出すとその方向を変えるのはとてもとても難しい。最も効率的でエレガントな方法で問題を解決する方法を、時間をかけて正確に決める必要がある。

V.　低く約束し、高く提供する

不愉快なサプライズではなく、ハッピーサプライズを提供しよう。低めに約束し（目標を控えめに）、高めに提供する（約束以上のものを提供する）ことで、：

- 自分自身、プロジェクト、チームに対する自信を深める
- 物事がうまくいかない場合に備えて、万が一の事態に備えておく。
- ポジティブで受容的な雰囲気を作り出す

もし、何か問題が起きたとしても、時間通りに終了し、皆がハッピーになる。もし、物事が本当に悪い方向に進んだとしても、予想していたものを提供できないかもしれないが、それでも、過剰に約束した場合よりはマシだろう！

VI.　反復する！増やす！進化せよ！

解決する価値のある問題のほとんどは、一塊で飲み込むには大きすぎる。どんな深刻なプロジェクトでも、問題を解決するために何らかの分解が必要になる。これはうまくいくが、それぞれの断片がどのように分析され、解決され、全体がどのように組み合わされるかに細心の注意を払う必要がある。体系的なアプローチがなければ、1つの大きな解決策ではなく、100の異なる解決策になってしまう。

VII.　軌道に乗る

おそらく、あなたには最終的なゴールがあるはずだ。仕事かもしれないし、ビジネスがそれに依存しているかもしれないし、次のグーグル、次のワールドワイド・ウェブ、次のシーベル／SAP／オラクルで世界に革命を起こそうとしているのかもしれない。

20人のチームを率いるシニア・プロジェクト・マネージャーであろうと、一人のウェブ開発者であろうと、これは当てはまる。

VIII.　変化を管理する

私たちは変化する世界に生きている。プロジェクトが進むにつれて、計画から逸脱しようという誘惑が抑えられなくなる。ステークホルダーは新しい "面白い "アイデアを思いつき、チームはあらゆる種類の穴を突き進み、当初のゴールは流砂の中の雪の結晶のような永続性を持つようになる。スコープクリープやドリフトはプロジェクト失敗の主な原因であり、成功したければ変更を管理・コントロールする必要がある。

これは、書き留めた唯一不変のプランがあって、それ以外のアイデアはすべて封じ込めなければならないということを意味しているのではない。あまりに柔軟すぎると、プロジェクトは乗り手のいない馬のように蛇行し、あまりに硬直的すぎると、ステークホルダーから初めて新しい要件を投げられたときに、プロジェクトはガラスのように粉々に砕け散ってしまう。

これを処理する最善の方法は、計画を立て、定期的に更新し、全員がそれに従い、同じ方向を向いていることを確認することだ。

IX.　早めのテスト、こまめなテスト

エラーを排除する唯一の方法はテストである。もちろん、ミスを防ぐために多くの貴重な作業

を行うことはできますが、人間にはミスがつきものです。

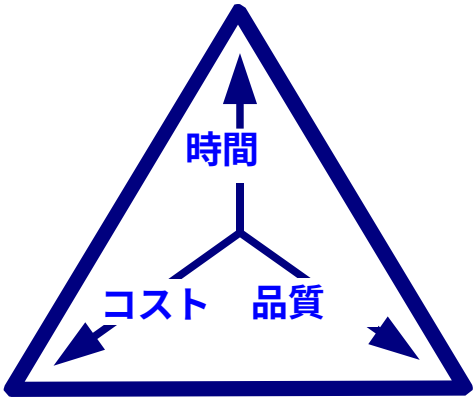
X. オープンマインドでいよう!

柔軟に対応すること! 本質的な成果は、結果に満足する顧客に完成したプロジェクトを納品することである。これを達成するためには、必要なあらゆる手段を用いることができ、上記のすべてのルールは、適切な状況、適切な理由であれば破ることができる。

イデオロギーに囚われるな。方法論に目を奪われるな。

プロジェクトの遂行に集中し、利用できるすべてのツールと人材を活用する。スケジュールに目を配り、状況に合わせて期待や計画を調整する。完成品を納品し、その使用を促進し、成功を祝い、次のプロジェクトに移る。

スコープ 三角形



スコープ・トライアングル」または「クオリティ・トライアングル」と呼ばれるこの三角形は、あらゆるプロジェクトに内在するトレードオフを示している。

この三角形は、プロジェクトにおける3つの主要な力の関係を示している。「時間」はプロジェクトを遂行するために利用可能な時間、「コスト」は利用可能な資金や資源の量、「品質」はプロジェクトを成功させるために達成しなければならない「目的適合性」を表している。

現実には、これらの要因のうち1つは固定され、他の2つは互いに反比例して変化するというのが通常の状況である。例えば、「時間」はプロジェクトにおいて固定されていることが多く、最終的なプロジェクトの「品質」は「コスト」や利用可能なリソースに左右される。同様に、「品質」のレベルが決まっているのであれば、プロジェクトの「コスト」は、利用可能な「時間」に大きく依存することになる（時間が長ければ、より少ない人数でできる）。

プロジェクトマネジメントの世界で「スコープクリープ」として知られる現象も、この三角形と関連づけることができる。スコープクリープとは、プロジェクトが新しい機能を追加していく、ほとんど止められない傾向のことである。しかし、スコープクリープが大量に発生すると、悲惨なことになります。

スコープがクリープし始めたら、増加したスコープをカバーするために新しい機能を追加しなければならない。これは三角形の品質アームで表され、ユーザーの要求を満たす「製品」の能力を表している。より多くの要求が満たされれば、より質の高い製品となる。

この状況では、選択肢は3つしかない：

1. 時間の追加 - プロジェクトを遅らせて、機能を追加する時間を増やす。
2. コストの追加 - 余分な仕事をするために人を募集、雇用、または獲得する。
3. 品質削減 - 必要でない要件と新しい要件をトレードオフする マネジメントの技術が意思決定にあるとすれば、プロジェクト・マネジメントの技術は次の点にある。

素早く決断するこのようなジレンマに直面したら、迷わず上記の3つの選択肢のいずれかを取るべきである。遅れると、プロジェクトが失敗するリスクが高まる。

> 勘のいい読者なら、2つのポイントが固定されるとどうなるのだろうと考えるだろう。通常、これはコストが固定され、納期が確定している場合に起こるもので、あまりにも身近な状況である!
>
> 機能を削るというのは思い切った手段のように思えるかもしれないが、経験豊富なプロジェクトマネージャーなら、まるでジャガイモの皮をむくように、喜んで機能を削っていくだろう。追加機能はいつでも "次のプロジェクト "に回すことができる。もしコア機能を提供しなければ、次のリリースはない。
>
> 本当に経験豊富なプロジェクトマネージャーなら、いざというときに犠牲にできるような余計な機能をプロジェクトに詰め込んでしまうかもしれない（でも、私には聞いてない！）。

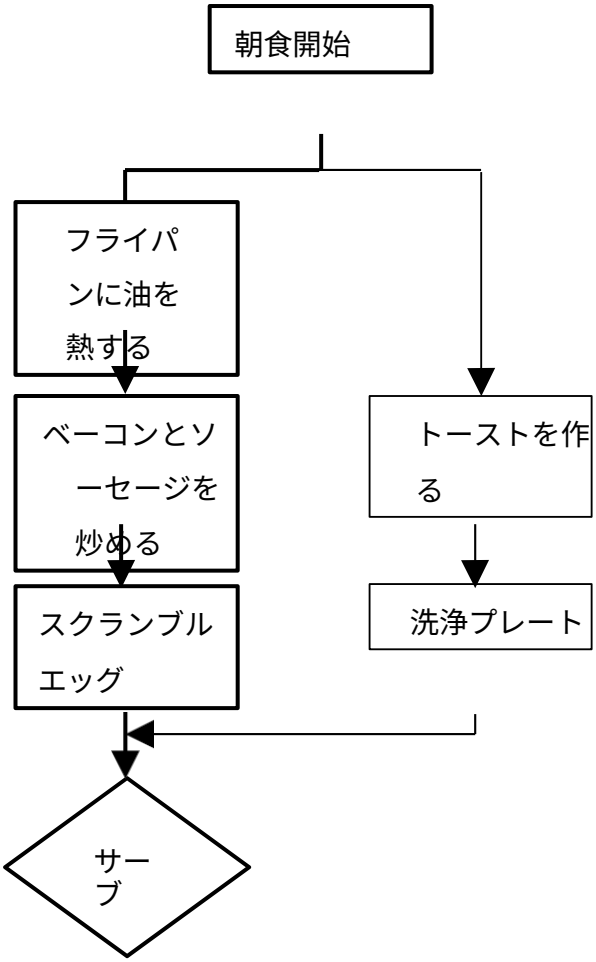
優秀なプロジェクトマネージャーは、1つまたは複数の軸をうまく活用し、プロジェクトの重点を他の軸の1つに移す必要があることを認識する。優れたプロジェクトマネージャーは、3つの軸を芋づる式にうまく使い分け、時間対品質対リソースのトレードオフを効果的に行う決断を日々下している。

クリティカル・パス

プロジェクト計画におけるもう一つの重要な概念は、クリティカルパスである。プロジェクトを完了させる必要のあるタスクの集合で構成する場合、クリティカルパスはそのようなタスクの集合の最小値、クリティカル集合を表します。これは矛盾しているように見えるかもしれません。

クリティカルパスは、プロジェクトで完了すべきタスクの理想的なセットではなく、最小限のセットを表している。プロジェクトを期限内に完了させるためには、このパスを通過しなければなりません。クリティカルパス上のタスクはプロジェクトの納期に影響するため、極端な場合のみ変更する必要があります。

次の例では、クリティカルパスを太字で表している。朝食を作るプロジェクトを完了するためには、ベーコンとソーセージを炒め、卵をスクランブルするというステップを踏まなければならない。



トーストを焼く」と「皿を洗う」というタスクは、重要ではあるが、他の3つのタスクほど時間に依存せず、クリティカルでもない。これらのタスクのどちらかを動かすことはできるが、クリティカルパス上の何かを動かそうとすると、プロジェクトが遅れてしまう。

朝食にトーストを食べるのが理想だが、a）必須ではなく、b）どの工程でトーストを焼くかは問題ではない。ベーコンを焼く前にトーストを焼こうが、焼いた後にトーストを焼こうが、全体の仕上がりにはほとんど違いがない。

その一方で、油が熱くなる前にベーコンを焼

くことはほとんどできないし、ベーコンを焼く前に卵をスクランブルエッグにすることもできない（糊になってしまう）。

クリティカル・パスは、プロジェクトを成功させるために*必要な*、重要な一連の出来事を表している。

通常、主要なマイルストーンはクリティカルパス上に表され、それらはしばしば発生する。
プロジェクトのさまざまな糸がひとつになるとき。

例えば、右の図では、私の唯一のマイルストーンは、完成した朝食を提供するときである。この時点で、私は準備を終え、両方のトラックのすべてを完了する。

突然仕事に遅刻しそうになっても、プロジェクトのオプションである "トースト "コンポーネントを陽気に破棄し、代わりにクリティカル・パスを取ることができる。

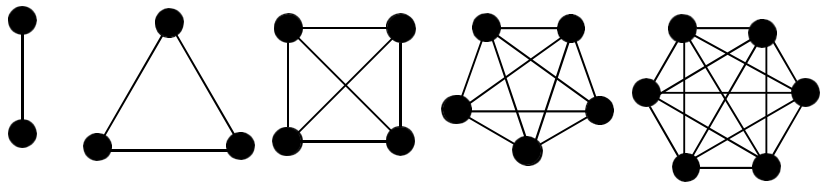
神話の男 今月

1975年、ソフトウェア開発の草創期に、フレデリック・ブルックスという人物が、このテーマについて多くの本や記事を書いた。彼の最も有名な著書は "No Silver Bullet（銀の弾丸はない）"であり、その中でブルックスは、ソフトウェア開発は、品質、コスト、複雑さといった様々な問題に対して、厳格な方法論を採用する以外に、雷を落とすような解決策は期待できないと指摘している。

銀の弾丸はない』にわずかに劣るほど有名なのが、ブルックのもう一つの論文『神話的人間月間』である。

ブルックスは「神話の人月」の中で、プロジェクトに人を増やしてもスピードは上がらないと論じている。リソースを増やせばソフトウェア製品の納期を早める*ことができる*のは事実だが、スピードの向上は追加されたリソースの量に正比例するわけではない。別の言い方をすれば、プロジェクトにリソースを追加するだけでは、納期を早めることはできない。

その主な理由は、人数が増えることでコミュニケーションの複雑さが増すからだ。プロジェクトチームに人が増えるごとに、コミュニケーションの複雑さは指数関数的に増していく。プロジェクトにはそれぞれ、人を増やすとかえって遅くなる損益分岐点がある。



人々	2	3	4	6	6	(n)
インターフェースs	1	3	6	10	15	$\frac{2^{-n}}{2}$

上の図は、この原則を図式化したものである。インターフェイスが増えれば増えるほど、コミュニケーションは複雑になり、プロジェクトにオーバーヘッドが増えることになる。

この計算が信じられないなら、論理的に考えてみよう。開発サイクルの間にプロジェクトに参加する人が増えるたびに、トレーニングを受け、現在の状況と割り当てられたタスクについて説明を受ける必要がある。

スケジュールが遅れたり、プロジェクトの納期が脅かされたりすると、真っ先に人を投入しようとする。このやり方がうまくコントロールされているプロジェクトではめったにうまくいかないし、うまくコントロールされていないプロジェクトでは*絶対にうまくいかない*。

プロジェクトに人を増やすと、彼らを管理するための「帯域幅」が必要になり、目の前のもっと重要な目標から目をそらすことになりかねない。

ブルックの「神話の男月間」から学ぶべきことはいくつかある：

1. 官僚的な大規模チームよりも、自律的な小規模チームの方が効率的なので、プロジェクトを管理しやすい大きさに分割し、各グループを何らかの境界線の中で働かせる。
2. プロジェクトに人を加えたいなら、その人たちをどのようにチームに導入するか、慎重に計画したほうがいい。彼らが生産的になるまでにタイムラグが生じ、チームの他のメンバーの生産性を低下させることさえある。スケジュールに "フラットスポット "を探して、これらの人材をチームに導入する。
3. スコープトライアングル」の選択肢のひとつが減ってしまった！プロジェクトのスコープが拡大した場合、オーバーヘッドが発生するため、プロジェクトに人を増やしてもメリットは限られていることがわかる！

私が携わったあるプロジェクトは、「神話的人間月間」の背後にある真実を、他のどのプロジェクトよりもよく物語っていた。

私は、クライアントである大手銀行を代表するコンサルタントのテスト・マネージャーだった。その銀行のシニア・マネジャーは、このシステムの成功に自分の名声を賭けており、プロジェクトを成功させるためにはどんな費用も惜しまなかった！世界最大級のITサービス会社である開発会社は、急遽、地元に人材がいなかったため、海外から設計チームを空輸してき た 。また、この銀行との初めてのプロジェクトを成功させるために、一流のプロジェクト・マネージャーも地球の裏側から空輸してきた。

プロジェクトが進むにつれて、計画はますます野心的になり、プロジェクトに参加する人員もどんどん増えていった。開発者は近隣の国からソフトウェア・エンジニアを呼び寄せ、週末に帰国させるようになった。地元スタッフはプロジェクトに迂回させられたが、彼らは元のライン・マネジャーに報告したままだった。

カオスだった。開発者たちは指示を待つばかり。グラフィックデザイナーは、ビジネスロジックも確定していない画面のインターフェイスをせっせとデザインしていた。少なくとも3つの異なるバージョンの仕様書が出回っており、どれが最新版なのか誰も知らなかった。

私たちの役割は、銀行に提供されたシステムの品質を審査することで、実質的には銀行に代わってシステムを受け入れることでした！すべてのリリースは、開発者によってテストされておらず、未完成のまま引き渡されたため、大きなバグを抱えたまま差し戻された。

タカラトミーの知識では、このシステムは私たちの関与が終わった後も立ち上げられることはなかった。プロジェクト全体にかかった費用は数千万ドルに上ったはずで、プロジェクトはお蔵入りになってしまった！

自分が何をしようとしているのかを知らなければならない。

これは当たり前のことのように思えるが、プロジェクトの初期段階における明確さの欠如は非常に一般的であり、多くの問題を引き起こす。多くのプロジェクトは、何を達成したいのかが漠然としていたり、明確に定義されていない状態でスタートする。限られた時間の中でプロジェクトを成功させたいのであれば、製品が達成すべき最終的な状態を決め、具体的な目標を設定する必要がある。

製品開発に対するこの「創意工夫」的なアプローチは、目を見張るようなユニークな解決策を生み出すこともあるが、失敗や不十分な結果に終わることの方が多い。また、私たちのほとんどは、時間や資源が無限にあるような環境で仕事をしているわけではない。

そのためには、さまざまな可能性のあるアプローチから最善の解決策を選択する方法が必要だ。このプロセスにおける最初の、そして最も重要なステップは、何をもって成功とするかを定義することである。そうすれば、成功の定義に照らしてすべての可能性を評価し、最も適合するものを見つけることができる。
これがなければ、我々は暗闇の中で撮影することになる。

目的を明確にすることに規律を守れば守るほど、成功する可能性は高くなる。

スコープ、ビジョン、 目標

スコープとは、プロジェクトが包含するすべてのもの、プロジェクトが完了するために達成されなければならないすべてのものを表す一般的な用語である。これは、あなたのビジョン、目標、要件を包含し、「プロジェクト提案書」、より低いレベルでは「商業仕様書」や「技術仕様書」などの文書に具体化される。

ビジョン」という言葉は、世界中の技術者、非技術者に戦慄を与える。そして、それは当然である。ビジョンとは通常、意味のないキャッチフレーズやマーケティングの戯言の集まりであり、顧客から最後の一銭まで搾り取るためではなく、企業が礼儀正しく利他的な理由のために存在していると人々を騙すためのものだからだ。

そもそもなぜそのプロジェクトをやるのか? プロジェクトをプロジェクトたらしめているのは、それが独立したタスク（またはタスクの集合）であり、意図した結果をもたらすという

事実だ。

プロジェクトの目的を一文で述べることができなければ、それはおそらくプロジェクトではない。

もしかしたら、それはビジネスや生き方のアイデアかもしれないが、プロジェクトではないかもしれない。プロジェクトとは、終着点が決まっていて、その終着点はあなたの「ビジョン」によって定義される。

その単一のビジョンは、小目標に分割されるかもしれないが、プロジェクトのすべての異質な部分を1つの企業にまとめるリンクとなる。それは、チームとステークホルダーに目的意識を与え、プロジェクトの成功を定義する。

ゴールは、ビジョンよりも少しレベルが低く、より具体的なものである。ゴールは、プロジェクトの全体的なビジョンを直接サポートするものでなければならないが、その定義を洗練させるものでなければならない。一般的に、ゴールは顧客またはビジネスによって設定され、プロジェクトの成功をどのように達成するかを定義する。ビジョンがプロジェクト全体を包含するのに対し、ゴールはプロジェクトの特定のセグメントの目標のみを指すこともある。

スコープ（scope）、ビジョン（vision）、ゴール（goal）という用語は、ほとんど互換性がある。ここに示した定義は、最も一般的に使われているものです。ここに示した定義は、最も一般的に使われているバージョンです。

インスピレーションとしてのビジョン

プロジェクト計画のさらなるステップでは、より具体的な目標が目指されるが、最初の目標は、プロジェクト全体を包含するような幅広いものでなければならない。

目標やビジョンもまた、インスピレーションを与えるものでなければならない。

最も安いシステムを、最も短い時間で提供する。

一方、「*市場で最高のセールス・マーケティング・システムを提供する*」というような目標は、チーム・メンバーや利害関係者の個人的な関与を刺激する可能性が高い。

クライアントやマネージャーから何度も呼び出され、"頼んだドキュメントはどこだ？"と言われ、"なんでまたこんなことをやっているんだろう？"と自分に言い聞かせる。- と自問自答したとき、あなたの答えはこうなる。

ビジョンは書き留めたり、石にしたりする必要はない。大組織ではそうすることが多い。大組織はそういうことを好むからだ。しかし、唯一重要なことは、あなた、あなたのチーム、ステークホルダーがビジョンの内容を正確に理解し、それに同意することだ。

やりすぎは禁物だ。今は、商業的な流行語の語彙力を鍛える時ではない。あなたがビジョンを信じ、それを使えば使うほど、あなたのチームや顧客を含む他の人々もそれを信じるようになり、成功する可能性が高まる。

あなたができる最も重要なことのひとつは、一緒に働く人たちに信頼を抱かせることです。懐疑的になるのは人間の性であり、プロジェクトが成功すると考えるよりも、失敗すると考える方がたいていの人にとっては簡単だ!

チームから利害関係者、小切手にサインする人に至るまで、誰もが自分が何をしているのか知っているという安心感を求めている。

多くの場合、ビジョンはエグゼクティブ・スポンサーや、プロジェクトを依頼したクライアントからあなたの手に渡される。彼らとの話し合いの中で、「*時間とお金を節約できるセールス・マーケティング・システムが欲しい*」*という*ような、プロジェクトに対する彼らの言い方が特異であることに気づくだろう。

要件のフィルターとしての目標

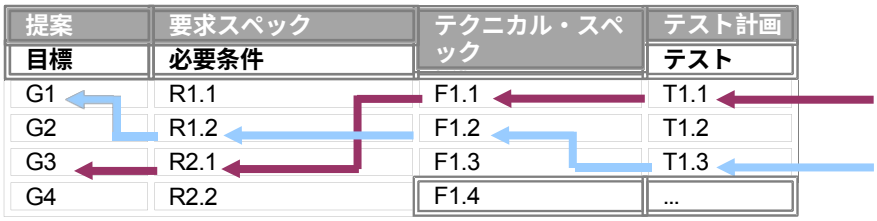
ゴールの主な目的のひとつは、後続の要求に対するフィルターとして機能することである。

もし特定の要件が、より高いレベルの目標からプロジェクト全体のビジョンまで遡ることができないのであれば、それはプロジェクトの範囲外となるため、取りやめるべきである。

例えば、プロジェクトの全体的なビジョンが「*市場で最高の販売・マーケティングシステムを提供する*」であった場合、適切なサブゴールは「*すべての海外拠点で使用できる販売注文処理システムを提供する*」かもしれない。請求書発行システムは財務システムの一部であり、販売システムではないため、不適切なサブゴールは「*請求書発行システムを提供する*」であろう。

このフィルタリングのプロセスは、プロジェクトのライフサイクル全体を通じて、追加機能の要求を評価し、プロジェクトに含めるかどうかを検討するために使用されるべきである。このようなフィルタリングのテクニックを使用すると、「スコープクリープ」の危険を避けるための簡単な方法を提供します。

低レベルの目標と高レベルの目標の結びつきは、チームにとっても基本的に重要です。これらをリンクさせることで、製品の目的を通して進捗を追跡することができます。これは*トレーサビリティとして*知られている。



まともな プロポーズ

ビジネスケース」と呼ばれることもあるプロジェクト提案書は、プロジェクトにおける最上位の目標を示すものである。顧客やクライアントが決定したプロジェクトの全体的なビジネスゴールやビジョンを概説するものです。プロジェクト開始のかなり前に作成されることもありますが、（運がよければ）作成に携わることもあります。

基本的な提案書には、プロジェクトのビジョンとビジネスゴール、つまりクライアントがビジネスレベルで達成したいことを記載する。また、記載されたゴールを修飾したり裏付

けたりするために、大量の裏付け情報があるかもしれないが、ゴールは明確でなければならない。

プロジェクト提案書は、商取引に合意した際に署名されるものであるため、しばしば重要な文書となる。そのため、プロジェクト提案書を慎重に検討する必要があります。

次のページでは、仮定のセールス・マーケティング・システムに基づいた例を示す。最初にビジョンを述べ、その後に具体的なビジネス目標と技術目標を列挙する。それぞれの具体的な目標は、販売・マーケティングシステムを提供するというビジョンに直接貢献するものである。

ウィズ・バン顧客関係揶揄システム

このプロジェクトは、市場で最高の顧客、販売、マーケティングシステムを提供するものでなければならない:

- *受注処理にかかる時間を50％削減（手作業による処理時間の場合）*
- *四半期ごとに詳細な経営報告書を提出*
- *要請に応じて詳細な市場・顧客分析を提供*
- *マーケティングROIを測定するために、売上をマーケティング施策に直接リンクさせる。*
- *顧客や見込み客の詳細情報を各アカウントマネージャーに提供する。*
- *ウェブサイトによる免許更新の完全自動化*
- *海外オフィス向けに、インターネット経由でアクセス可能なゼロフットプリント・クライアントを提供する。*
- *他の販売注文システムのユーザーにアップグレードパスを提供する。*

これは長くもなく、過度に詳細でもないことにお気づきだろう。上記で概説した目標は、おそらくかなりの量の商業調査や市場調査によって裏付けられるだろうが、プロジェクトの文脈の中では、上記はプロジェクトの目的を確立するのに十分すぎるほどのものであるはずだ。

また、目標によってはより具体的なものもあることに注意する必要がある。例えば、*「販売注文の処理にかかる時間を50％短縮する」*というのは、かなり具体的で、すぐにテスト可能な目標です。一方、*「四半期ごとに詳細な管理レポートを提供する」*というのは少し漠然としています。どのような形式で? 誰のために?

大まかな目標は示されたので、それをどのように達成するかを定義するのが（要求仕様のような）後続のフェーズの目的である。

プロジェクト提案書には十分な時間をかけ、正確で簡潔なものにしましょう。これは、上級管理職がプロジェクトの成否を判断する際の基準となります。時間をかけすぎて、プロジェクトの開始を遅らせてはいけません。

投資利益率

多くの正式なビジネスケースの重要な構成要素の一つは、「投資利益率」またはROI計算である。簡単に言えば、ROI計算は、製品のコストと期待される利益を比較するものである。そうすることで、様々なプロジェクトを同程度に評価することができ、その中から最適なものを選択することができます。

コストと便益を比較するためには、明らかに同じ「単位」で記載する必要があり、驚くことではないが、これらは通常「ドル」である。プロジェクトのコストを決定するのは通常かなり簡単だが、便益を定量化するのははるかに難しい場合がある。ベネフィットは通常、顧客がその製品を使うことでどのようなベネフィットが得られるか、ドル換算で見積もるよう求めることで決定される。

手作業のプロセスから誰かを解放する（その結果、経費を節約する）ことから、ビジネスにより多くの顧客を引きつけること、既製品の単純な販売まで、さまざまな可能性がある。これらはすべて、金額で表すのはかなり簡単だが、すべて将来の予測に基づいているため、信頼できない可能性がある。例えば、誰かの1日を何時間空けることができるか? この商品によって何人の顧客が増えるのか、あるいは初年度に何件の売上を達成したいのか。

とはいえ、これらの値のいずれかを自信を持って見積もることができれば、次のように比較することができる。

投資利益率は、コストと利益の比率に過ぎず、比率がプラスであれば利益、つまり投資に対するリターンを示す。これは通常、製品がいつ「回収」ポイントに達するかを判断するために時間をかけて行われる。

例えば、マーモセットの繁殖に役立つ素晴らしい新製品を作ると決めたら、ROIを次のように計算することができる：

1. 製品の設計、開発、デバッグにはおよそ6週間かかると予想している。また、約2000ドルの新しいPCと約500ドルの重要なマーモセット測定器も必要だ。また、私自身には1日約500ドルの報酬を支払うので、そのコストは次のようになる：30営業日×500ドル＋ハードウェア2500ドル＝17500ドル

2. 私の地元には約1000人のマーモセット・ブリーダーがいる。私の名声が広まるにつれて、そのうちの30%に販売することができ、その後、競合製品が市場に出回るにつれて、売上は年間10%程度に減少していくと予想しています。

   マーモセットのブリーダーは裕福ではないので、私は自分のソフトウェアに1部50ドルほど課金することにした：

   1000の10%は100×50ドル＝初年度5000ドル 1000

   の30%は300×50ドル＝2年目15000ドル

   1000の10%は、3年目以降は100×50ドル＝5000ドルとなる。

3. 私のROI計算はこのようになる：

	年目	年目	3年目	年
コスト	$17500	$0	$0	$0
ベネフィット	$5000	$15000	$5000	$5000
ROI	28%	114%	143%	...

この計算に基づくと、私の製品は1年10カ月で回収できることになる。たった6週間で書いたにしては悪くない！ちょっとマーモセットを買いに行ってきます。

必要条件

要求仕様とは、プロジェクトの目標を絞り込み、クライアントを満足させるために何を達成しなければならないかを決定するプロセスである。

プロジェクトが正式に開始される前に、ソリューションや技術に適したものを特定し、選択するために、要求仕様の策定が行われることもある。多くの場合、これはフィージビリティスタディまたはプロジェクト分析として知られています。しかし、より一般的には、いくつかの要件は（おそらく提案書の中で）一緒に投げかけられ、本当の要件仕様は、プロジェクトが開始した後にのみ発生します。

機能要件

機能要件とは、エンドユーザーや利害関係者が製品に対して持つ、日常的に必要とされる明らかな要件のことである。

これは「アサーション」として知られている。アサーションは、ステークホルダーの目から見て、システムまたは製品に必要な、または望ましい振る舞いを主張または断言するものである。

明確な主張がなければ、要件は漠然とした議論に過ぎず、机の上も心も散らかってしまう残念な傾向がある。

次の 2 つの対照的な機能要件を比較してください:

- *財務システムは、付録Aに従った詳細な顧客請求書を作成しなければならない。*
- *顧客への請求書の作成は重要である。インボイスには、顧客が商品の代金を支払うために必要なすべての関連情報が含まれていなければならない。*

1つ目は、アサーションとして記述された機能要件です。これは、*金融システム*が、*付録A の*すべての情報を含む*詳細な顧客請求*書を作成する責任があることを示しています。もっと具体的にすることもできますが、読者は、金融システムが成功する金融システムであるために何を*しなければならない*かについて、疑う余地はありません。

もうひとつは、会計の本の序章のようなものである。請求書が重要であることは述べられているが、誰が、あるいは何が、請求書を作成する責任があるのかについてはまったく示されていない。このような記述は要求仕様書にはふさわしくない。

つ目の『要件』は、一見しっかりしているように見えて、実は曖昧で曖昧であることによって問題を複雑にしている。*適切な情報とは*何を意味するのか? *顧客が支払いを行えるようにすることは*、そのための請求書なのだから余計なことである。この文章は正確ではあるが、システムが成功するために何をしなければならないかについての理解には何の役にも立たない。

以下は、より "良い "要求事項の記述である：

- *顧客アカウントレコードには、一意のアカウント参照、主要な連絡先名、連絡先詳細、および過去売上年度内の顧客に対するすべての売上リストが含まれていなければなりません。*
- *連絡先の詳細は、電話番号、住所、Eメールアドレス（任意）で構成される必要があります。*
- *各連絡先について、最大5つの電話番号に対応する必要がある。*

非機能要件

これらは「非機能要件」と呼ばれるが、私の考えでは、これはちょっと矛盾している。

パフォーマンス	パフォーマンスには通常、応答性、スループット、動作速度などが含まれる。
使いやすさ	完成品はどの程度「使いやすい」のか? 例えば、体の不自由なユーザーやハンディキャップのあるユーザーに対応しているか? しかし、一般的な使いやすさを考慮すべきです。 わかりにくい、あるいは複雑なインターフェース。
信頼性	信頼性要件は、ユーザーに対する製品の*継続的な可用性を*扱うものである。
セキュリティ	機密情報や機微な情報を扱う製品では、セキュリティを考慮する必要がある。さまざまなレベルのアクセス、暗号化、保護に関する要件を収集する必要がある。
財務	プロジェクトの成否を左右する財務上の考慮事項があるかもしれない。例えば、銀行や投資家は、プロジェクト期間中に満たさなければならない特定の財務上の制約や条項を指定するかもしれません。
Legal	製品が使用される環境により、満たさなければならない法的要件があるかもしれません。これらについては法律の専門家にご相談ください。
オペレーション	日常的な運営上、考慮しなければならない問題が数多く存在する可能性がある。これらに対応できなければ、プロジェクトの立ち上げが遅れることはないが、立ち上げ後のエンドユーザーによる利用が制限されたり、停止されたりする可能性がある。
スペシャリスト	どのプロジェクトにおいても、プロジェクトの性質や事業の性質に依存する専門的な要件が数多く存在します。これらは個別に検討し、設計書の中で明示する必要があります。

ステークホルダー

ステークホルダーとは、エンドユーザーやクライアントのことであり、要件が引き出される相手であり、設計に影響を与える相手であり、最終的には完成したプロジェクトから利益を享受する相手である。

ステークホルダーをプロジェクトの全フェーズに参加させることは、2つの理由から非常に重要である：第一に、経験上、利害関係者がプロジェクトに関与することで、自己修正フィードバックループが構築され、成功の可能性が大幅に高まること、第二に、利害関係者をプロジェ

クトに関与させることで、製品への信頼が構築され、ターゲットとするオーディエンスに受け入れられやすくなることです。

ステークホルダーにはさまざまなタイプがあり、それぞれに異なる対応が必要である：

エグゼクティブ	エグゼクティブ利害関係者は、請求書を支払う人々である。通常、彼らはプロジェクトの商業的目標に関与するマネージャーや取締役である。 彼らの経験やスキルは、"典型的な "エンドユーザーとは大きく異なるからだ。
エンドユーザー	あなたの製品を使う人たちです。その製品がデスクに置かれたときに何をすることになるのか、彼ら以上に知っている人はいない。誰も！あなたを含めて！あなたは自分がよく知っていると思っているかもしれないが 彼らの言うことを聞かなければ、自分をごまかしていることになる。
専門家	他分野の専門家の意見が必要な場合もある。グラフィックデザイナー、サポート担当者、セールス、時には弁護士や会計士などだ。

要件 キャプチャ

要件収集とは、利害関係者の生の要件を収集し、それを有用なものに変えるプロセスである。これは、基本的に彼らのニーズを決定するために利害関係者の質問です。これは、アンケートやインタビューが最も一般的であるなど、多くの形態を取ることができます。通常のアウトプットは「要求仕様」文書であり、プロジェクトがどのステークホルダーの要求に対応し、重要なことは、どの要求に対応しないかを詳細に記述する。

要件捕捉の焦点は情報収集にある。耳を開き、口を閉じる！ 製品の設計を始める前に、人々が何を望んでいるのか注意深く耳を傾けよう。物事を*どのように*実現するかは後で考えるとして、今は*何を*実現しなければならないかを知る必要がある。(設計の決定には仮定が必要であり、その結果、利害関係者を「誘導」し、彼らが望む製品ではなく、あなたが望む製品を提供することになりかねない）。

> 技術者は、ステークホルダーはしばしば「自分が何を望んでいるのかわかっていない」と不平を言う。問題なのは、ステークホルダーがあなたのためにシステムを設計することができないということであり、彼らが望んでいることを実現する方法をあなたに伝えることができないということである。要求仕様のコツは、ステークホルダーが与えてくれるものを、あなたが彼らの望みを実現する方法を決定するのに役立つものに抽出することである。これを考える1つの方法は、利害関係者の現在の問題に対する理想的な解決策を見つけることである。

要件把握もまた、迅速である必要がある。プロジェクトは、この段階で行き詰まり、膨大な量のドキュメントを作成する傾向があるが、有用なアウトプットはない。要求の取り込みの目的は、ありとあらゆる質問に対する答えの詳細を延々と書き連ねることではなく、プロジェクトチームに十分な明確さを提供し、目的を明確にすることである。

アンケート

アンケートは、ステークホルダーから要件を収集する典型的な方法である。標準的な質問セットを使用することによって、プロジェクトチームは、みんなのニーズに関するいくつかの情報を収集することができます。アンケートは、迅速に多数の要件を収集するために効率的ですが、それは一方通行のプロセスであるため、その有効性は制限されることがあります。これを解決するために、アンケートは通常、インタビューなどの他の方法と組み合わせて使用されます。

インタビュー

インタビューの利点は、トピックをより深く掘り下げ、プロジェクトの要件についてオープンエンドなディスカッションができることです。これは双方向のプロセスである。

インタビューは、単独またはグループセッションとして構成することができる。特定の利害関係者に個別にインタビューすることもできるし、グループセッションで多数の利害関係者の間でアイデアを出し合うこともできる（できればコンセンサスを得たい）。グループセッションでは、正式な構造を使用するか、またはアイデアを投げるよりオープンなスタイル、ブレーンストーミングセッションを使用することができます。

インタビューのマイナス面は、時間と人手がかかることである。インタビューの設定と実施だけでなく、議事録の作成、配布、レビューが必要であり、フォローアップミーティングを1回以上開催する必要があるかもしれない。質問*票と*インタビューを使用することは、一般的で効率的な方法である。質問票は事前に配布し、利害関係者の要求の概要を収集することができる。

ユーザー観察

要件を捕捉するもう1つの方法は、エンドユーザーによる直接的な観察または評価である。

ユーザ観察の目的は、エンドユーザが意識していないかもしれない要件を把握することである。例えば、大規模な財務システムで小切手処理システムを使用している人は、自動化できるはずの多くの手動ステップをシステム外で行っている可能性がある。彼らはこれらのステップをシステムの一部とみなしていないため、現行のシステムについて質問されても、それについて言及することはないだろう。直接観察することによってのみ、開発チームはこれらのステップの重要性に気づくことができる。

自由形式の観察を使うこともできるし、典型的なエンドユーザー・グループに一連のタスクを設定し、彼らのタスク処理をモニターすることもできる。彼らがタスクを遂行する方法、彼らが遭遇する障害についてメモを取り、（彼らが同意すれば）ビデオテープに録画することもできる。

ユーザーを直接観察する方法は、最初の2つの要件捕捉方法とは異なり、観察された事実に依存し、意見に依存しないため、特に強力である。しかし、3つの手法の中で最もリソースを必要とする。

相反する要件

1つまたは複数の要件に、他の要件の要素と直接競合する要素が含まれている場合があります。例えば、性能要件は、基幹システムをリアルタイムで更新しなければならないことを示すかもしれないが、（他の要件によって定義された）システムのサイズとスコープがこれを妨げるかもしれない。そのような大規模なシステムをリアルタイムで更新することは不可能かもしれない。

要件間の矛盾を解決する最も効果的な方法の1つは、要件に何らかの形で優先順位を課すことである。これは、矛盾する要件を評価するための基礎を提供するため、交渉の可能性を可能にする。例えば、前の例から、リアルタイム更新の要件が、完全な顧客データを含めるよりもはるかに高い優先順位で評価された場合、妥協に達することができる。メインの「オンライン」データベースは、顧客の詳細（リアルタイム更新を可能にする）の必要最小限のものだけを含むことができ、顧客の履歴を含む別の「アーカイブ」データベースを確立することができる。

多くの場合、要件は他の要件と密接に絡み合っており、利害関係者の見解が正反対で折り合いがつかないように見える。どのような要件であっても、それをバラバラにすれば、両者とある種の妥協点を見出し、コンセンサスを得ることができる。これは困難で、感情的なプロセスになることさえある。

相反する要件を解決するために私が見た最も良い方法は次のようなものだ:

1. 利害関係者が要求事項のリストを作成する
2. このリストは、最も重要なものから最も重要でないものまで、厳密な優先順位で整理されている。
3. 次にプロジェクトチームは、スケジュールを見て、利用可能な時間とお金で提供できると思われるものに基づいて、リストに線を引く。
4. 利害関係者は開発状況を評価し、必要に応じて要求の優先順位を変更したり、要求の性質や重要性について交渉したりすることができる。
5. コンセンサスが得られれば、双方がサインオフし、作業が開始される。

しかし、これはコンセンサスを得るための単純で簡単な方法ではない。要件リストの基本的な順序付けでさえ、規模の大小にかかわらず、プロジェクトでは並大抵のことではない。

要件の文書化

プロジェクトチームのために、要件を文書化する方法が必要です。また、利害関係者は、自分たちが望んでいることの確認として、要件に署名するよう求められることも多い。多くの場合、ここで金銭の授受が始まる。

要求仕様書は、プロジェクト全体の設計において不可欠なリンクであり、プロジェクトの全体的な目標に意味を与えようとするものである。

どのような形式の要求文書であれ、*どのような*決定がなされたかだけでなく、*なぜ*そのような決定がなされたかを網羅すべきである。決定に至った理由を理解することは、繰り返しを避けるために非常に重要である。例えば、単に実現不可能であるという理由で特定の機能が除外された場合、その事実を記録する必要がある。もしそうでなければ、開発中やテスト中にステークホルダーからその機能の復活を要求されたとき、プロジェクトは無駄な作業や繰り返しのリスクを負うことになる。

意思決定プロセスを文書化することは、利害関係者の観点からも有用である。なぜなら、利害関係者は、最終製品に何を期待すべきかをよりよく理解することができるからである。根本的な議論がない基本的な要求事項の記述は、素人やエンドユーザーにとって理解するのが難しい場合がある。

SMARTの要件

要件を定義する際に便利な頭字語の1つに、SMART があります：

具体的 目標や要件は具体的でなければなりません。解釈が曖昧にならないよう、明確な言葉で表現する必要があります。また、簡潔で、余計な情報を避ける必要があります。

測定可能 要件には測定可能な結果がなければならない。

達成可能 要件やタスクは達成可能であるべきで、現実的に達成できない要件を設定しても意味がない。

関連 要求仕様書には、厳密には必要以上の情報が含まれていることが多く、それが文書化を複雑にしている。簡潔で首尾一貫したものにすること。

テスト可能 要求が満たされていることを証明できなければなりません。テスト可能でない要件は、プロジェクトが宙ぶらりんの状態になり、納品され

たことを証明することができません。

要求仕様書の言語とレイアウト

要求仕様書には多くの情報が含まれていることが多い。私は、特定の決定が下された理由を説明するために、文脈に沿った情報を含めることを提唱しているが、議論と「主張」を簡単に分離する方法が必要である。

そこで私は、次のような経験則を適用している：

- 各要件には「アサーション」が必要である。
- アサーションが文書化される場合は、製品が「しなければならない」または「すべき」ことを述べた一文で構成されるべきである。

- 「Must "は必要条件、"should "は "nice-to-have "を示す。
- アサーションとディスカッションを区別する簡単な（視覚的な）方法が必

要である。 以下に、ディスカッションとアサーションを*イタリック体*で強調した例を

示す：

> **1.1 ユーザビリティ** - システムのユーザビリティは、システムの採用にとって非常に重要であると考えられている。クライアントからは、ユーザビリティ・テストを実施するために必要な期間について懸念が出されましたが、プロジェクト・チーム全体としては、ユーザビリティ・テストを含む開発フェーズの時間延長を支持しました。
>
> *システムはシンプルで使いやすく、会社のデザインハンドブックに記載されている標準的なUIスタイルに従わなければならない。*

この場合、ディスカッションは将来の参照用に保存されるが、要件は本文から明確に区別される。仕様書を読むプロジェクト・メンバーは、仕様書を読み飛ばし、すばやく簡単に要件を引き出すことができる。文書を読むマネジャーも同じことができるが、決定の背景を確認して、それがどのように生まれたかを理解することもできる。

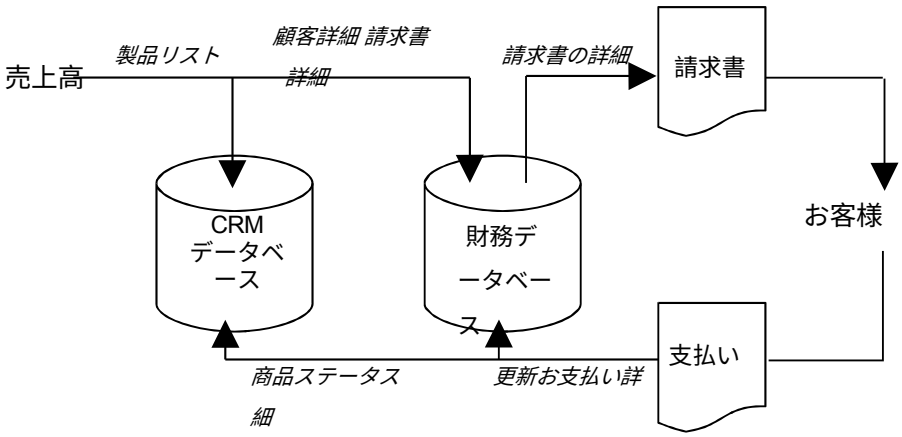
> 要件の分類は、MoSCoWのルールを用いて行われることもある：
>
> 必需品はプロジェクト成功の基本である
>
> あるべき姿は重要だが、プロジェクトの成功はそれらに依存するものではない。
>
> 可能性のあるものは、プロジェクトに影響を与えることなく簡単に省くことができる。
>
> 今回のラウンドは除外し、後日実施することもできる。
>
> 私は、"should have "と "could have "の区別は決して明確ではなく、通常、クライアントとプロジェクトマネジャーの間で多くの議論の対象になると感じているので、通常、"could have "は省略する。そうすると、すべての要件は、必要（"must have"）か任意（"should have"）のどちらかになります。優先順位をつけることにこだわる。

ダイアグラム方式

この章では、テキスト形式の要求仕様に重点を置いてきたが、よりグラフィカルな方法も利用できる。絵や図で情報を表現することは、非常に有益である一方で、非常に混乱させることもあります。絵や図で情報を表現することは、非常に有益である反面、非常に混乱させることもあります。図には、実際に記述することなく要件を暗示したり、詳細を解釈する余地が残されていることがよくあります。

このような理由から、グラフィカルな方法は、標準的なテキストによる記述方法をサポートするものであると私は考えている。グラフィカルな表現が要求の本質をよりよく表すことができる場合には、適切な場合にはどこでも使用すべきです。特定の構造やプロセスの性質を言葉で説明するのが難しい場合は、ぜひ適切な図を挿入してください。

簡潔で正確な表現に重点を置かなければならないので、グラフィックとテキストの要素をどのように組み合わせてもよい。



要求仕様書のサンプル

最も一般的な方法は、文書やマニュアルで使用されるようなアウトライン方式で要件を分解することである。

例えば、こうだ:

1. **現在の製品ステータス** - すべての関係者が、製品の現在のステータスを明確かつ公に示す指標の必要性を強調した。
2. **日付** - 各主要マイルストーンの日付も必要であると認識された。これらの日付のいくつかはパブリックドメインに残るが、他の日付は "プライベート "ユーザーにのみ公開される。プライベート・ユーザーは、適切と思われる日付を公表することができる。

   指定された日程は以下の通り:

   2.1 開発サインオフ

   2.2 テストのサインオフ...

要件カテゴリを分割することにより、文書にさらに構造を持たせることができます: 例えば

1. 機能要件

1.1. **製品リスト** - *販売中または開発中の製品のリストを作成する。*

1.2. **現在の製品ステータス** - すべての関係者が、*製品の*現在のステータスを明確かつ公に*示す*指標の必要性を強調した。

1.3. **日付** - *各製品について、各主要マイルストーンの日付を示さなければならない。*これらの日付の一部はパブリックドメインに残るが、他の日付は「プライベート」ユーザーにのみ公開される。*プライベートユーザーは、適切と思われる日付を公表する能力を持つべきである。*

関連する日程は以下の通り:

1.3.1. デザイン・サインオフ

1.3.2. 開発サインオフ

1.3.3. テスト・サインオフ

1.3.4. その他...

2. 非機能要件

2.1. **パフォーマンス** - *システムは毎日更新され、本社が情報を掲載してから1分以内に、すべての海外ユーザーが情報を利用できるようにしなければならない。*

2.2. *システムはシンプルで使いやすく、会社のデザインハンドブックにある標準的なUIスタイルに従わなければならない。*

2.3. **セキュリティ -** *スケジュール情報へのアクセスはユーザーごとに管理する必要があります。*

可能な限り具体的に書くのがベストですが、第10の戒めを忘れずに、柔軟に対応しましょう。もし、あなたのプロジェクトがこのレベルの詳細を必要としないのであれば、それを含めなければよい。あなたと組織のニーズに合った、詳細と労力の中間を見つけましょう。

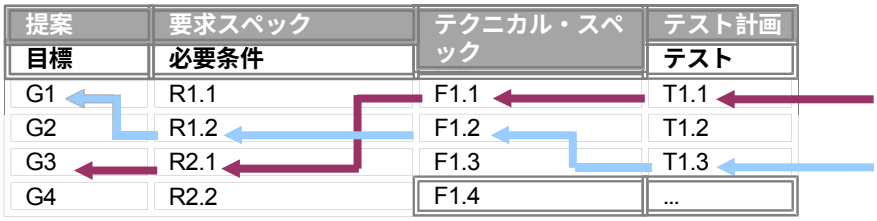
トレーサビリティ

最も時間のかかるオーバーヘッドのひとつは、トレーサビリティの管理である。

何百、何千もの利害関係者の要求がある、それなりに複雑なプロジェクトの場合、一つの要求が満たされているかどうかを、仕様から設計、開発、そして最終的にはテスト、ローンチに至るまで、どのように追跡するのだろうか。

テスト時や実装時に、要件が満たされていることをどのように証明するのか? 開発中、特定の要求の達成状況をどのように追跡しますか? また、設計が以前のフェーズで指定された特定の要件をすべて組み込んでいることを、どのようにして確認するのでしょうか?

最も単純な方法は、上記の例ですでに概説したとおりである。要求仕様と技術仕様の両方に共通のフォーマットを使用することで、要求から設計要素までを比較的簡単に追跡することができます。テスト計画も同じフォーマットに基づいていれば、トレーサビリティをテストフェーズまで拡張することができ、どのテストがどの設計要素を検証し、結果としてどの要件を検証したかをテストレポートに示すことができます。



しかし、これだけでは十分でないことが多い。大規模なプロジェクトでは、要求事項の数が膨大になるため、このようなフォーマットでは対応しきれなくなる。また、1つの要件が設計の複数の要素で満たされていたり、設計の1つの要素が複数の要件を満たしていたりすることもあり得ます。このため、単純な参照番号による追跡は困難です。

しかし、この場合、メンテナンスのオーバーヘッドが大きくなり、親ドキュメントと同期させるのが非常に難しくなります。書式と構造によって、すべての要件に対して1対1のトレーサビリティを提供できるわけではありませんが、適切な構造を使用することで、混乱を最小限に抑え、より複雑なソリューションの必要性を排除することができます。

このようなシステムの利点は、トレーサビリティの問題を強調するレポートを自動的に作成できることである。

ペラペラペラ

らないか。これは、高いレベルではスコープ文書であり、低いレベルでは要求仕様書である。

*いつまでに行う必要が*あるか-目標を達成しなければならない期限

*誰が*や*るのか*- 「リソース」という不親切なレッテルを貼られることもある人材、あるいはその目標を達成するためのチームである。

どのように達成するのか- これは通常、技術仕様書などの文書に記載されている。

通常、企画書、スケジュール、予算の草案は、プロジェクト開始前に1つずつ完成させます。通常、企画書、スケジュール、予算の草案は、プロジェクト開始前に1つずつ完成させます。その他の文書は、ライフサイクルのある時点で完成させます。また、どの文書も静的で不活性な原稿と見なすべきではない。それぞれは、特定の時点におけるプロジェクトの生きた表現であり、プロジェクトの発展とともに進化していくものです。

最後に、これらの文書にはどれも、義務的なサイズ、形式、長さはありません。さまざまな形式やスタイルを提案していますが、それらは厳密なテンプレートというよりはむしろ例です。これらは教義ではなくツールであることを忘れないでください！ カクテルナプキンの裏にプロジェクトスケジュールを走り書きする方が簡単で効率的なら、そうすればいい！ 重要なのは目的であり、形式ではない。正しい形とは、あなたにとって機能する形だけなのです。

スケジューリングの妙技

なぜスケジューリングに "芸術 "が必要なのか?

もしそれが科学であれば、すべてのプロジェクトは納期通りに納品されるはずだ!

悲しいことに、これは事実ではないようだ。実際、オーバーランはあまりに一般的になったため、人々はプロジェクトの期限に対する信頼を失い、大きな皮肉をもって見るようになっている。

実際、スケジュール管理の技術は経験に基づいており、経験を積めば積むほど、より正確なスケジュールを立てることができるようになる。しかし、簡単なルールを守るだけで、正確なスケジュールを作成することはできる。

スケジューリングの原則

ルールその1: 決して、その場しのぎや考えなしの返答をしない
こと（実現できないことを約束しないこと）。

スケジューリングは、ある部分では予測であり、ある部分では期待管理である。すべての要素を評価する時間があるときに考慮された応答は、はるかに優れています.あなたがランダムな会議で "その場で "日付を選択するように圧力をかけられた場合、日付は間違っているだけでなく、それはあなたを悩ます戻ってくることは間違いありません。

> クライアントや経営陣とのミーティングで、経験の浅いプロジェクト・マネジャーが「今日から4週間後」というような日付を口走るのを、私は何度も見てきた。私の知る限り、彼らのうち誰ひとりとして納品した者はいない。それはプレッシャーへの対応であり、喜ばせたいという願望だった。
>
> もし経営陣から迫られたら、現実的な答えがあればそれを言うか、適切な見積もりを立てるためにもう少し時間をもらうこと。

ルールその2: できる限り不確実性を排除する。

プロジェクト計画において、より具体的であればあるほど、スケジュールはより正確なものになります。もし、機能性やその他の項目を計画で特定しないままにしておくと、せいぜいスケジュールで概算することしかできないでしょう。しかし、やりすぎは禁物です。バランスというものがあります。もし、プロジェクトの納期に何の影響も与えないタスクに詳細を追

加することに時間を費やしているのであれば、それはおそらく時間の無駄です。

ルールその3：変動に対応できるよう、十分な不測の事態を想定する。

プロジェクトがどんなに綿密に計画され、スケジュールがどんなに正確であっても、不可避のランダムな影響によって、慎重に練られたスケジュールが台無しにされることはある。人が病気になったり、設備が故障したり、外的要因が陰謀を企て、目標期日に間に合わなくなったりするのだ。予期せぬ遅れに対処できるよう、十分なコンティンジェンシー（不測の事態）を想定しておく必要がある。

また、コンティンジェンシーはプロジェクトのタイムライン全体に分散させるべきであり、最後にだけ置くべきではありません。もし、偶発事象のプールをプロジェクトの最後に1つだけ割り当てると、大きな不確実性を残すことになります。それを分割し、プロジェクト全体に分散させることで、より多くの選択肢が生まれ、プロジェクトをより綿密にコントロールできるようになります。また、未使用のコンティンジェンシーをプロジェクトに戻せば、時間を「買い戻す」こともできる。

コンティンジェンシーとは、第五の戒め、つまり「約束は低く、納期は高く」を意味する。見積もりで悲観的になり、予想以上のパフォーマンスで人々を驚かせる方が、見積もりを骨抜きにし、最初の災難でスケジュールが吹っ飛ぶよりましである。「数字を考えて、それを2倍にする」という古い格言は、プロジェクトのスケジューリングに関しては、ある程度有効である。

ルールその4 - 適切な粒度レベルを選ぶ

スケジュールを作成する際には、適切な詳細レベルを選ぶことが重要だ。チームから毎日アップデートを要求するのであれば、1日ごとに区切るのが理にかなっている。そうすれば、いつまでに何を達成しなければならないかを全員が同じように理解できる。

一方、もしあなたのプロジェクトが、同じような活動、たとえばテストに多くの時間を割いているのであれば、単純に1ヶ月か2ヶ月のテストをブロックスケジュールしたほうがいいかもしれない。詳細はチームに任せてもいいかもしれません。

つまり、タスクは週単位でスケジューリングされるということだ。週単位の方が、月曜や火曜にタスクを終わらせるよりも、週明けに終わらせる方が自然に思えるからだ。

日ごとのスケジューリングは、本当に必要な以上のオーバーヘッドを引き起こす可能性がある。水曜日に完了する予定だったタスクが、困難な状況により完了できなかった場合、チームメンバーがそのように予測したとしても、木曜日に完了する可能性は低い。それよりも、2、3日オーバーして金曜日に終わる可能性の方が高い。一日単位でスケジュールを組むと、スケジュールの更新にばかり時間を取られ、プロジェクトの管理に手が回らない。一方、週単位でスケジュールを組むと、このような小さな変動に対処するのがずっと簡単になる。もし「21日の月曜日から始まる週」に予定されていたものが1日、2日、あるいは3日遅れたとしても、その後のタスクは無理なく移動させることができるし、まったく影響を受けないかもしれない（私のコンティンジェンシーのレベルにもよるが）。

唯一の例外は、プロジェクトのペースを強制する必要がある場合だ。しかし、コントロールのレベルが高ければ高いほど、注意のレベルも高くなる。スケジュールが細かくなればなるほど、私は個々のタスクに細心の注意を払い、確実に完了させる必要がある。

ルールその5：不測の事態に備えてスケジュールを立てる

プロジェクトマネジメントとは、未知の事態に対処する技術である。予期せぬ出来事や状況がプロジェクトの流れを妨げることはよくある。それらをすべて受け入れるのがあなたの仕事

だ。最も起こりそうな遅延を想定してスケジュールを立て、万が一発生した場合にはそれに対処する。経験や直感から、ある種のタスクがオーバーランすることがわかったら、それを予測し、いくつかのコンティンジェンシーを用意し、それが発生したときに十分なリソースを手元に確保しておくこと。

これに対処する良い方法は、即席のリスクマネジメント（「リスクマネジメント」を参照）を実施することである。起こりそうなリスクを予測し、優先順位をつけることで、予期せぬ事態に対処しやすくなる！また、プロジェクトのリスク評価を他の誰かに任せることも、大いに意味がある。

マイルストーン

スケジュールとは、時間的な順序を示すものである。プロジェクトでは、「マイルストーン」は通常、プロジェクトの重要な部分の完了、またはあるフェーズから別のフェーズへの移行を示します。各マイルストーンでは、次のフェーズに進むために完了しなければならない「成果物」があります。

前述したように、スケジュールには合理的に実用的な粒度を含めることができます。実際、複雑なプロジェクトでは、さまざまなレベルのスケジューリングが行われるかもしれません。あなたは、高いレベルのマイルストーンを持つ週間スケジュールで仕事をするかもしれませんが、他のチームメンバーは、より細かいグレードのマイルストーンを持つ小規模なスケジュールで仕事をするかもしれません。

この場合、クリティカルパスの概念が登場し、マイルストーンは、「ブランチ」がプロジェクトのクリティカルパスに再び合流する重要なイベントを表す。

簡単な例

設計-開発-評価という通常のサイクルが、1月22日に始まり3月1日に終了する1つの "プロダクション・フェーズ "に連結されている。

マイルストーン	完成
プロジェクト開始	01月
スコープとプランの完成	1月10日
要件を指定する	1月21日
生産量	3月1日
実施内容	3月8日
レビュー	3月15日

この例は極めて単純であり、一人以上のプロジェクトがこの方法で成功す る 可能性は低いだろう。単純であるため理解しやすいが、コントロールツールとしてはあまり役に立たない。スケジュールが極めてハイレベルであるため、プロジェクトチームがプロジェクトが軌道に乗っていないことを示す唯一の兆候は、主要なマイルストーンを逃したときだけである。

また、タスクとタスクの間にコンティンジェンシー（不測の事態）が予定されていないため、プ

ロジェクトが複雑化したり、納期に間に合わなかったりする可能性がある。すべてが計画通りに進むと仮定することで、プロジェクトは、何か問題が発生した場合に、自分たちで操作する余地をほとんど残していないことになる。

より複雑な例

以下の、より現実的な例では、プロジェクトをいくつかのフェーズに分け、それぞれのフェーズの開始と終了を別々に記録しています。また、各フェーズの期間と、そのフェーズの終了時に完了する成果物を記載する欄が追加されている。

この新しいプロジェクトのスケジュールには、設計-開発-評価の生産サイクルの3つの反復が*予定されている。*これらの生産サイクルの成果物は、システムの「アルファ版」、「ベータ版」、「最終版」である。

フェーズ	スタート	終了	期間	納品物
範囲と計画	01月	10-1月	7日	プロジェクト提案
要求事項の指定	1月13日	1月24日	10日	要求スペック。
生産フェーズ1	2月3日	2月14日	10日	アルファ
生産フェーズ2	2月24日	3月7日	10日	ベータ
生産フェーズ3	3月17日	3月28日	10日	最終候補
受け入れテスト	4月7日	4月11日	5日	リリースシステム
実施	4月21日	4月25日	5日	-
打ち上げ	4月28日	4月28日	0日	-
合計			67日	

まず注意しなければならないのは、「期間」欄が、予定された開始日と終了日の間の暦日数と一致しないことである。これは、「期間」欄が、2つの日付間の*営業*日数を示しているためである。例えば2003年の場合、1月1日から1月10日まで10日間あるが、1日は元旦で祝日であり、5日と6日は週末であるため、営業日は7日間しかない。

次に注目すべき重要な点は、プロジェクトの各フェーズが、もはや連続したものではないということだ。つまり、プロジェクトのいくつかのフェーズの間には明確なギャップがある。たとえば、1月24日に終了した要求仕様の策定と2月3日の最初の生産フェーズの開始の間には、5営業日の空白がある。

物事が計画通りに進まない可能性が高い場合、プロジェクトはこのコンティンジェンシーを使ってスケジュールと当初の納期を維持することができる。万が一、計画通りに物事が進まなかった場合、プロジェクトはコンティンジェンシーを廃止し、スケジュールを次のフェーズに進めることができる（その結果、納期が短縮される）。いずれにせよ、誰もがハッピーなのだ!

コンティンジェンシーをどの程度含めるかは、経験の問題である。コンティンジェンシーは納期を大幅に膨らませる可能性がある一方で、プロジェクトにダメージを与え、コストのかかる失敗を避けることもできる。上記の例では、コンティンジェンシーは製作期間全体の50%近くに

もなり、プロジェクト側の過剰な警戒心を表している可能性があります。経験則から言えば、10〜20%が普通ですが、それ以上でも珍しいことではありません。

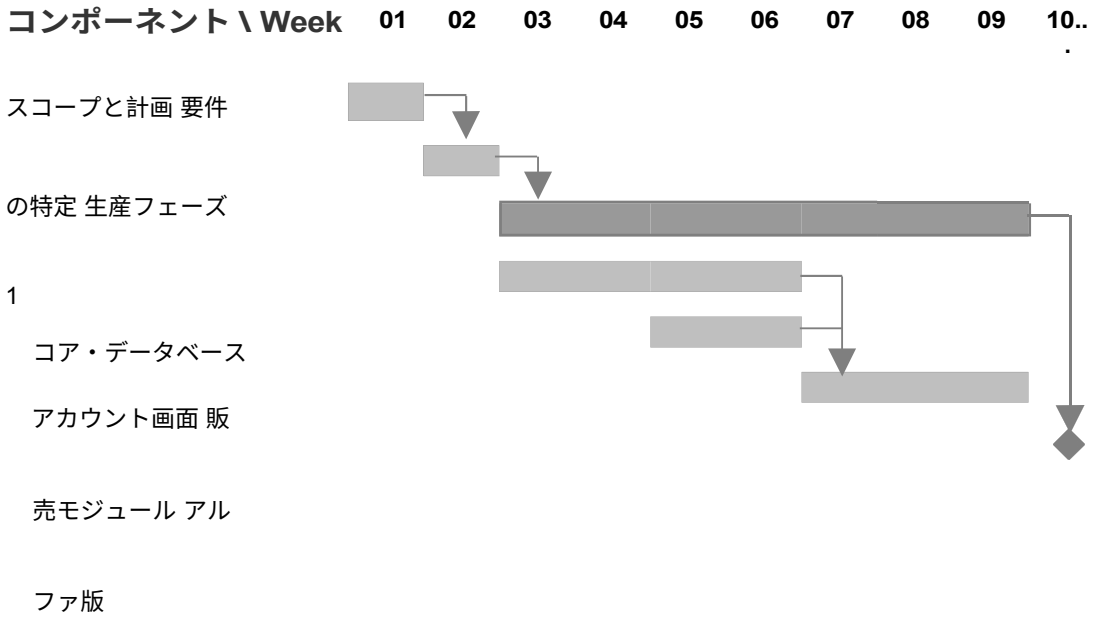
不測の事態の問題点のひとつは、それを意識したとたんに、自然とそれを利用したがるということだ。デザイナーも、開発者も、テスターも、たとえあなたのスケジュールがそれを許さないとしても、最高の仕事を確実に提供するために、利用可能な最大限の時間を利用したいと思うだろう。

不測の事態が存在しないふりをすることはできない。どうせチーム全員がそのことを知っているのだから。真の緊急事態のために予定されている時間を邪魔しないよう、彼らのプロ意識に訴えかける必要がある。

不測の事態を未然に防ぐ効果的な方法のひとつだが、軽視されがちなのが、納期厳守に対する成功ボーナスを提供するか、より適切には、納期遅延に対する金銭的ペナルティを導入することだ!

ガントチャート

ガントチャートとは、スケジュールを視覚的に表したものである。ガントチャートでは、時間は横軸で表され、タスクは左側にリストアップされる。タスクの期間はグラフの本文で横棒で表される。タスク間の依存関係もリンクされた矢印で示される。矢印は、各タスクに必要な完了順序を示し、したがってプロジェクトの進捗（クリティカルパスを含む）を示す。



上記の図では、"プロダクション・フェーズ1 "が3つのサブ・フェーズに分割されていることがわかります。コア・データベース"、"アカウント・スクリーン "そして "セールス・モジュール "の納品は、最初のプロダクション・フェーズの全体的な見積もりとアルファ・バージョンのマイルストーンの納品のために、すべて別々に見積もられています。コア・データベース "と "アカウント・スクリーン "については、（並行して進行しているため）別の人が作業しているか、誰かが残業しているかのどちらかであることに注意してください!

最も精巧で美しいガントチャートを作るプロジェクトマネジャーが、プロジェクトを予定通りに完了できるとは限りません。ガントチャートは、スケジュールを設計したり、監視したり、伝達したりするための道具である。ガントチャートは、スケジュールを設計したり、監視したり、伝達したりするための道具である。

特に、マルチカラーやマルチレイヤーのガントチャートを作成するプロジェクトマネージャーには注意してください。また、依存関係がスパゲッティのようなガントチャートを作成する人も要注意である。多くの場合、このような天才的な作品は、プロジェクトマネジャーが不心得者の気をそらす、あるいは、だますための簡単な方法である。

経営陣が迷路のようなガント・チャートを見ながら頭を悩ませている間、プロジェクト・マネージャーは神のみぞ知るプロジェクトにハイ・テールで向かっていることだろう。

ガントチャートがそれだけでプロジェクトを成功させたことはない!

原価計算と 予算編成

スケジューリングがアートだとすれば、コスト計算はブラックアートと言えるかもしれない。原価計算が比較的簡単なプロジェクトもあるが、そうでないものも多い。人件費単価のような単純な数字でさえ、計算するのが信じられないほど難しく、ほとんどの場合、概算が使われる。

会計、原価計算、予算管理はそれ自体が広範なトピックであるため、ここではそのすべてを網羅しようとは思わない。その代わりに、原価計算と予算編成のプロジェクトへの具体的な適用に焦点を当て、必要な用語と原則の基礎を学んでいただこうと思います。

留意すべきいくつかの基本原則は、会計実務から導き出されたものである：

- 慎重さ」の概念-決算では悲観的になるべきである（「利益が出ないことを予想し、起こりうるすべての損失に備える」）。これは昔から言われていることだ。
- 発生 "の概念-収益と費用は互いに発生または一致し、スケジュール内の同じポイントに帰属する。例えば、請求書を支払う時点でハードウェアのコストが予算に計上されている場合、請求書を受け取った時点でハードウェアのすべてのコストが「発生」しているはずです。
- 一貫性」の概念：これは発生主義と似ているが、異なる期間における一貫性を重視するものである。特定の費用の計上基準を変更した場合、それに合わせて過去の財務会計をすべて修正するか、あるいは、人々が同質基準で比較できるように、変更点を適切に注記する必要がある。

原則は優先順位の高い順に記載されていることに留意されたい。一貫性の原則が慎重さの原則と対立する場合は、慎重さの原則が優先される。

原価計算

基本的なレベルでは、原価計算のプロセスはいたってシンプルである。考えられる支出をすべてリストアップし、各項目に数値をつける。

有形費 用

- 資本支出 - プロジェクトで購入した大型資産。通常、工場、ハードウェア、ソフトウェア、

時には建物などが含まれるが、これらは様々な方法で会計処理することができる。

- リース費用 - 一部の資産は、そのまま購入するのではなく、プロジェクトの耐用年数にわたって費用を分散させるためにリースされる。これらの資産は、プロジェクトまたは会社が所有するものではないため、資本支出とは別に会計処理されるべきである。
- スタッフ・コスト - スタッフにかかるすべてのコストを計上しなければならない。これには、給与および年金（退職年金）コスト、保険コスト、採用コスト、スタッフの雇用、訓練、維持に直接結びつくあらゆるものが含まれる（ただし、これらに限定されない）。
- プロフェッショナル・サービス - すべての大規模プロジェクトは、弁護士や会計士など、1人または複数のプロフェッショナル・グループの意見を必要とする。この分野の支出を注意深く監視する必要があるため、これらは通常、別に計上される。精査を怠ると、コンサルタント・エンジニア、会計士、弁護士のコストは、あっという間に他のコストを凌駕してしまう。
- 消耗品および消耗品-消耗品に対する定期的な支出は、予算の単一項目でカバーするのが最善であることが多く、その項目でこれらの数字が計上される。

- 単発費用-単発費用は、上記のカテゴリーとは関係なく、不定期に発生する支出に適用される。これを人件費に計上するのが適切かもしれないが、不定期費用として単独で計上したい場合もあるだろう。選択は自由だが、慎重さと一貫性の原則が適用される。
- 間接費（Overheads）-間接費と呼ばれることもあり、上記のカテゴリーのいずれにも直接帰属しないが、予算に影響を与えるコストである。例えば、プロジェクトの電話代をスタッフコストに反映させるのは適切ではないかもしれませんが、それでも支払わなければなりませんし、計上しなければなりません。オーバーヘッドのコスト計算は、通常、「スタッフコスト」のような他の要素に対する大まかなパーセンテージで行われます。

無形費用

プロジェクトに対する貢献の中には、非常に価値のあるものもあるが、必ずしも有形の価値を持つとは限らない。では、それらを予算や原価計算に計上すべきなのだろうか? 慎重さ」の原則は「イエス」だが、現実的には「ノー」である。このような会計上の曖昧な分野に踏み込むのであれば、専門家の助けや助言を求めるべきです。

無形資産で予算に計上される典型的なものは、「のれん」や「知的財産」である。無形資産の源泉としてよく使われるのは人事関連であり、無形資産の貸借対照表には「経営陣」、「人間関係」、「人脈」などが記載されている。

予算編成

正確なコスト・モデルを作成するためには、もちろん、かなり詳細な設計、あるいは最低でも要求仕様書を作成する必要があります。

最初の見積もりには細心の注意を払い、常に「約束は低く、納品は高く」の戒律に従わなければならない。

原価計算と予算編成は、プロジェクト内の他のタスクと同様に、反復的なライフサイクルに従います。設計を改良すればするほど、それに基づく原価計算も改良する必要があります。

スケジューリングと同様に、予期せぬ支出を考慮し、十分なコンティンジェンシー（予備費）を組み入れる必要がある。例えば、クリティカルパスの不具合により、タスクが遅延し、マイルストーン（ソフトウェア購入など）の期限が予定した*翌月に迫ってしまった*場合、入念に計画したキャッシュフローが狂ってしまう可能性があります。しかし、プロジェクトを慎重に予算化していれば、変動は比較的簡単に発見でき、その都度対処できるはずだ。

スケジューリングと同じように、定期的に予算の見直しを行い、財務状況とこれまでの支出を検証し、それに応じて計画予算を調整する必要がある。

リスク マネジメント

リスクマネジメントは、最近のプロジェクトマネジメントには欠かせない言葉のひとつである。リスクマネジメントの概念を理解することは、あなたにとって役に立つだろうが、それが適切に使われているのを私はほとんど見たことがない。

リスクマネジメントの基本的なコンセプトは、プロジェクトがうまくいかなくなる可能性のある重要なものを特定するプロジェクトの「チキン・リトル」アセスメントである。通常、プロジェクトマネジャーやビジネスマネジャーは、プロジェクトがうまくいかなくなったときの「掟」のようなものとしてリスクマネジメントを使う。

リスクマネジメントの最も良い使い方の一つは、プロジェクトの定例会議で「リスク」という項目を設け、プロジェクトが現在直面しているリスクを全員に強調することである。

リスク管理オフィサー

リスクマネジメント担当者の責任は、プロジェク トのすべてのリスクを特定し、優先順位をつけてプロジェクトチームに提示し、 解決することである。小規模なプロジェクトでは、リスクマネジメントはプロジェクトマネジャーの役割に内在し ていることが多く、個別に検討されることはない。しかし、このセクションで説明する原則は、小規模なプロ ジェクトチームでも活用することができる。リスクをプロファイリングし、解決するという公式化されたプロセスを、プロジェクトに 対する潜在的な脅威を計画するための非公式な方法で適用することができる。

リスクプロファイル

プロジェクトのリスクは、失敗の*可能*性と失敗の*影響の*2つの軸で測ることができる。

問題が発生する可能性が高ければ高いほど、それがプロジェクトに与えるリスクも高くなる。かなり些細な問題や課題であっても、回避できないほど頻繁に発生すれば、プロジェクトにとって脅威となる可能性がある。同様に、問題の影響や結果も重要である。問題の中には、それだけでプロジェクトを止めてしまうものもある。

リスクをさまざまなカテゴリーに分類するシステムは数多く存在するが、ここで紹介するものはかなりシンプルである。このシステムでは、各リスク項目は「*可能性*」と「*影響*」*という*2つの尺度で評価される。それぞれの尺度は「低い」「高い」に分けられ

、それぞれの尺度に従ってリスクが評価される。

重大な」リスクは、その頻度や影響の重大性から、プロジェクトにとって重大な脅威となるものであり、通常、可能な限り早急に対処しなければならない。
可能性が高くもなく、特に深刻でもない。しかし、些細なリスクは、背を向けたときに大きなリスクに変わるという厄介な癖がある。

リスクの解決

リスクのプロファイリングが完了したら、プロジェクトにとって対処すべき様々な脅威を表すリストにランク付けすることができる。

典型的な行動は以下の通りだ:

研究	リスクがまだ十分に理解されていない。その影響や発生の可能性が不明確であったり、発生する可能性のある文脈が不合理と思われたりする可能性がある。プロジェクトチームのメンバーによる更なる調査は 保証する。
受け入れる	リスクは避けられないものであり、そのまま受け入れなければならない。このカテゴリーのリスクは、プロジェクトにとって非常に重要なものとなる。そのため、予測は次のようになる。 このカテゴリーのリスクに対処する鍵である。
リスクを減らす	プロジェクトチームは、リスクを低減し、リスクが発生した場合の緊急対応策を確立するために行動しなければならない。リスクは将来的に見直し、それがもたらす脅威を明確にする必要がある。
排除する	リスクはどのような状況においても容認できないものであり、可能性として排除しなければならない。プロジェクトチームは、当面の脅威を確実に排除するだけでなく、以下のようなプロセスや手順を導入しなければならない。 今後、再発することはない。

リスクの記録

リスクを記録する最も簡単な方法は、リスクとその優先順位をリストアップした表やスプレッドシートを用いることである。これをプロジェクトチームが定期的にレビューし、リスクを軽減または排除するための適切なアクションをとることができる。

以下は、あるプロジェクトのリスクテーブルの例である:

説明	頻度／影響	重大性	タイムフレーム	アクション
主要チームの短期欠場メンバー	ハイ／ロー	メジャー	プロジェクト	受け入れる
主要チームの長期欠場メンバー	低い/高い	メジャー	プロジェクト	削減
サプライヤーXが製品を納入しないY	低い/低い	マイナー	フェーズ1	受け入れる
誤ったスケジューリング	高/高	クリティカル	プロジェクト	削減

…	…	…	…	…

上の表では、サプライヤーがいくつかの部品を納入できないことは、軽微なリスクと評価されている。このような判断は、経験に基づいて、現在のプロジェクトの文脈の中でしかできない。サプライヤーがよく知られ信頼されている場合、納期が遅れる可能性は低く、したがってリスクは軽微と分類できる。

スケジューリングをプロジェクトリスクの重要な領域と位置づけるのも、経験の結果である。もし、過去に同じような性質のプロジェクトが、スケジュール上の問題で頓挫したことがあれば、このプロジェクトも同じような問題に見舞われる可能性が高い。プロジェクトマネジャーが、どんなに正直であっても、自分のスケジュール管理能力を「ハイリスク」と評価することは考えにくいからです。

変更 マネジメント

大きな長さのプロジェクトであれば、状況に応じて当初の計画から外れることは避けられない。もし変更が管理されず、気まぐれに起こるのであれば、それはもはやプロジェクトではなく、無政府状態である!

変更管理とは、潜在的な変更の影響を評価し、プロジェクトへの影響を管理する方法です。例えば、クライアントの要求が変更された場合、小さな修正で済むかもしれませんし、設計の全面的な書き直しを意味するかもしれません。変更管理は、これを評価し、管理された方法で変更を導入するプロセスを提供します。

変化は不可避であるため、プロジェクトへのインプットを処理する流動的な方法が必要である。プロジェクトのインプットである要件や設計が、変化に対応し、時間とともに進化していくことが重要です。もしインプットが静的で変更不可能な文書であれば、プロジェクトにおける状況の変化に対応することができず、足かせとなるでしょう。

> 変更管理の最も重要な点は、何が変更されたかを実際に知ることができることである。
>
> ある製品では30人以上のプログラマーと仕事をしたが、実際の変更管理はなかった。プログラマーたちは毎日ソフトウェアへの変更をチェックし、毎晩、150万ものデータファイルを処理し、約500行のテストレポートを作成する巨大な自動テストを実行していた。
>
> ある日、出社すると、一晩で10〜20%も成績が向上していた。翌日、出社すると、最初の30分で製品がクラッシュして使い物にならなくなっていた。問題は、誰が、何に責任を負っているのかわからず、変更管理もされていなかったことだ。
>
> 最終的にはシステムを導入し、目標に向かって着実に前進することができた。

変更管理プロセス

チェンジ・マネジメントの基本は、誰もが理解できる明確なプロセスを持つことである。それは官僚的であったり、煩雑であったりする必要はないが、普遍的に、好意を恐れずに適用されるべきである。変更プロセスの基本的な要素は次のとおりである:

- 何が変更管理の対象で、何が対象外なのか?
- 変更はどのように要求されるのか?
- 誰が変更を承認または拒否する権限を持っているのか?
- 承認または不承認の決定はどのように文書化され、周知されるのか?

- 変更はどのように実施され、その実施はどのように記録されるのか?

そのプロセスは、広く理解され受け入れられ、官僚的であったり杓子定規であったりすることなく、効果的でなければならない。プロジェクトチームがクライアントのニーズに応えていると思われることが重要であり、過度に悪意ある変更管理プロセスほど、これを損なうものはない。
プロジェクトにおいて変化は避けられないものであり、それをコントロールする必要は

あるが、抑制することは避けたい。典型的なプロセスは次のようなものだ:

1. プロジェクト文書が利害関係者によってサインオフされた後、その文書に変更を加えるには、電子メールを介して記録される必須の*変更要求が*必要です。この要求には、変更の性質、変更の理由、変更の緊急性の評価が含まれます。
2. 開発マネージャー、テストリーダー、プロダクトマネージャーで構成される「変更管理委員会」が問題を評価し、各変更要求を承認または却下する。より詳細な情報が必要な場合は、変更管理委員会のメンバーが依頼内容を調査し、報告する。

3. 変更要求が1週間以上残ってはならない。
4. 重要または緊急の要請は、変更管理委員会の最も近いメンバーを通じてエスカレーションされるべきである。
5. 受理された各変更は、毎週の開発会議で討議され、グループによって対応方針が決定される。その後、開発チームのメンバーは、それぞれの担当分野で変更を実施するよう割り当てられる。

柔軟な変更要求プロセスがあれば、チームメンバーは、プロジェクトチーム全体に問題を伝えることが有用と思われる場合、追加情報や明確化を求めるために、そのプロセスを利用することが奨励される。

変化の追跡

変更管理を容易にするためには、変更を追跡、評価、記録するシンプルな方法が必要だ。これは単純なデータベースやログでもよいが、大規模なプロジェクトでは、それ自体がカスタマイズされた情報システムに発展している。

そのため、システムは対応できる必要がある：

- 製品および文書に対する変更要求の記録
- 特定の変更の優先順位の記録と管理
- 変更管理権限の決定を記録する
- 変更の方法と実施を記録する
- 製品や文書の特定のバージョンに対して実施された変更の追跡

システムを構造化すればするほど、変更管理プロセスの安全性は高まるが、オーバーヘッドが増えるのは明らかである。

変更管理システムは、全員の期待を管理するためにも有用である。多くの場合、意思決定は利害関係者や顧客から要請されるものであり、適切な協議が行われないと、（要請されたからといって）自動的に自分たちも含まれると思い込んでしまうことがある。

変更要求の量が特に多い場合（多くの場合そうである）、何が変更され、何が変更されたかを手作業で伝達するのは困難である。シンプルでよく理解された変更管理システムは、多くの場合、利害関係者が変更とその承認を記録、追跡、レビューするために直接使用することができる

。

多くのプロジェクトでは、変更管理システムは、欠陥追跡システムとリンクすることができる (または、欠陥追跡システムの一部である)。この統合システムは、バージョン管理システムに含まれる製品またはシステムの特定のバージョンに対して、ソフトウェアの変更を直接参照する。

路線を維持

プロジェクトの実行段階における唯一最大の問題は、軌道に乗り続けることである。最高の計画を立て、最高のチームを編成し、ありとあらゆる落とし穴を予測しているにもかかわらず、プロジェクトには予期せぬ問題が発生するコツがあるようだ。

完成の神話

Microsoft Projectのようなプロジェクトツールによって宣伝されている、タスクに関するよくあるデマがある。そのデマとは、タスクは部分的に完了することがある、つまりタスクは10%でも20%でも完了することがある、というものである。

目標を達成したか、していないか。白か黒かの二者択一の命題だ。もし、あなたのゴールが「指導文書を書く」というような曖昧で不正確な意思表示であれば、それはあなたが始めた瞬間に完了する。

一方、タスクが明確に定義され、測定可能な成果物がある場合は、それが提供されるまで目標は達成されない。例えば「タスクが完了するのは、文書が作成され、レビューと編集が行われ、出版準備が整ったときだけである。

タスクを部分的に完了させることができると考えることの危険性は、誤った安心感を与えることである。タスクの50%というのは定義が難しいので、人はそのタスクに割り当てられた時間の50%を終えた時点で、そのタスクの50%を完了したと言う。タスクの90%は残っているかもしれないが、50%の時間が残っているのだから仕事も半分しか残っていないと主張する。

このような誤解は、「時間は伸縮自在だ」と信じている人たちが特に抱くものだ。つまり、特定の時間の長さの中にどれだけでも仕事を詰め込むことができる:

1. 20日間のタスクに10日間取り組んだ後、彼らは「まだ10%しかやっていない（つまり、10%しかやっていないと言った方が聞こえがいい）」「時間を取り戻す」と発表する。
2. タスクの前半に90%の時間がかかり、タスクの残り半分に残りの90%の時間がかかる」という古い格言は、往々にして真実である。

タスクを小さなステップに分割し、その完了を報告するだけにした方がずっといい。そうすれば、あるタスクが完了日を過ぎてしまった場合、自動的にスケジュールの調整が行われる。

人を管理する

プロジェクト・マネジャーがピープルマネジャーでもなければならないことは、あまり認識されていない。多くの場合、プロジェクトマネジャーは特定の技術的な流れを汲むものであり、正式なマネジメントのトレーニングを受けることなく、リーダー的なポジションに昇格することになる。

そのため、ほとんどのプロジェクトマネジャーは、スケジューリング、設計、テストなどのプロジェクトマネジメントの技術的な側面を得意としています。このセクションでは、プロジェクトマネジャーにとって重要なピープルスキルの概要を説明します。

交渉

交渉は技術者にとって厄介な仕事である。私たちは世界を白か黒かの二元的な環境として見る傾向がある。技術者」はしばしば、問題を解決するには正しい方法と間違った方法がある、あるいは一つの技術やソリューションが「最良」であると信じている。これは完璧を求める気持ちの一部ではあるが、実際には問題を解決する方法はたくさんあり、それぞれの技術やソリューションには長所と短所がある。

交渉とは、対立を避けながらコンセンサスを得るプロセスである。その中心となるのは、問題に対する最善の解決策は、関係者全員のコンセンサスを集めるものであるという理解である。一方的な（二元的な）解決策は、誰かを疎外したり失望させたりするため、定義上、最善の解決策ではない。最良の解決策を見出すには妥協が必要であり、プロジェクトマネージャーは、さまざまな関係者間の話し合いの支点となる。

つまり、「勝ち」あるいは成功するためには、誰かが「負け」なければならない。つまり、「勝つ」、あるいは成功するためには、誰かが「負ける」必要があるのだ。例えば、セールスマンは、誰かに高い値段で商品を買わせることができれば「勝ち」だと考えるかもしれない。これはゼロサムの態度であり、セールスマンは「勝ち」、顧客は「負け」である。

一方、顧客は価格を気にしないかもしれないし、製品に適切な機能があれば喜んで支払うかもしれない。もしセールスマンが、顧客が何を望んでいるのかを理解できれば、適切な商品を売ることができるかもしれない。さらに、特定の製品にそのような機能がない場合、セールスマンは価格を下げたり、顧客に購入を納得させるような他のインセンティブを提供できるかもしれない。セールスマンがこれを達成すれば、*両者ともに*勝利となる。

これが交渉術だ。

問題はいくつかの要素に分解することができると理解することで、別々に処理した場合、トレードオフが生じ、それによって「Win/Win」ソリューションを達成するこ と が できる。

これには信念の飛躍が必要だ。交渉は人を打ち負かし、自分の思い通りにするためのトリッキーな方法だと考えて話し合いに臨めば、必ず失敗する。人はバカではないし、ほとんどの人はあなたの外見的な態度に関係なく、あなたの意図を見分けることができる。

交渉は信頼の基礎の上に成り立つものでなければならず、もしあなたが信頼されないと思われるのであれば、あなたのアプローチがいかに巧妙であっても意味がない。オープンな態度で話し合いに臨み、合意に至るまでやり抜く。その後、その解決策について自分がどう感じたかを評価してみよう。そうでないなら、中古車のセールスマンにでもなればいい。

理解する

話し合いに参加している各当事者が、話し合いから何を得ようとしているのかを理解しなければなりません。話し合いが個人のグループで構成されている場合は、個人の*目標とグループ*の目標（異なる場合がある）を理解しなければならない。

また、自分が何をテーブルに持ち込み、何を譲歩する用意があるかを理解しなければならない。何を「交換」しなければならないかを知ることで、オープンマインドと柔軟性をもって話し合いに臨むことができる。理想的には、交渉に入る前にこれを知っておくべきだが、そうできない場合もある。

共感

共感とは、関係者の感情を理解することである。人間として、私たちは本能的な感情レベルで物事に反応し、好むと好まざるとにかかわらず、それが私たちの反応の多くを決定する。交渉の場にいる個人の基本的な感情を理解することで、単純な論理よりも直接的なレベルで相手に訴えることができる。

信頼

交渉の成功は信頼の上に成り立つ。信頼なくして真の妥協はあり得ず、したがって解決策もない。もし、あなたが提示する譲歩が不誠実であったり、不本意なものであれば、他の関係者からも同じように不本意な反応が返ってくるでしょう。多くの場合、関係者間の信頼関係を確立し、主導権を握るのは、極めて重要な立場にあるプロジェクトマネジャーの責任である。プロジェクトマネージャーは、基準を設定することで、交渉の基礎と交渉の基調を確立する。問題があることは理解しているので、ここにいる全員が解決に貢献することを期待している」というような言葉は、しばしば効果的である。

貢献

議論は暴力的な感情を避け、理性に基づいて行われるべきである。問題は明確に述べられ、合意され、解決策が提示されるべきである。話し合いのこの段階では、*すべての当事者が*貢献することが不可欠である。沈黙は許されない。全員から意見を求め、懸念を表明し、対処し、できれば解決すべきである。これを怠ると、1人または複数の当事者が、権利を奪われたと感じたり、幻滅したりすることになる。

コンセンサス

さまざまな解決策について話し合ったら、可能性のある解決策を提案し、それがすべての人に受け入れられるものであれば、前進させることができる。そうでない場合は、何らかの形で妥協策を打ち出し、各当事者は解決策を見出すために自分たちの要求の要素を譲歩する用意がなければならない。そして各当事者は、妥協案を受け入れて前進する意思を示さなければならない。

おなかをくすぐられるために寝転がったり、足を突っ張ったりしても、Win-Winの状況にはなりません！ コンセンサスを得るために、すべての譲歩をしなければならないと思って交渉に臨んではいけない。これでは、自分が弱く無力であると感じることになる。

これは「勝ち負け」と呼ばれる結果である。

その場にいる全員が、それぞれの問題についてギブ・アンド・テイクを行い、適切な結果を得るために協力してくれることを期待して交渉に臨むべきである。自分自身の期待値の最低レベルを設定し、それを達成するまでそのプロセスに満足してはならない。

チーム作り

このセクションのタイトルに苦笑されるかもしれないが、プロジェクトマネジメントの最も重要な側面のひとつが「チームビルディング」である。よく会社で見かけるようなチームビルディングのゲームではなく、もっと繊細なピープルマネジメントのスキルのことだ。

信頼 - オープンで正 直 で あれ

プロジェクトもオフィスと同様、商業的あるいは政治的な圧力によって、さまざまなレベルの情報開示が行われる秘密主義的な場所になることが多い。しかし、プロジェクトのうわさ話もオフィスと同じように素早く機能するものであり、もしあなたが誰かを秘密にしていたり、欺いていたりすれば、「希望退職」と言うよりも早く、あなたはオフィスのうわさ話の片棒を担ぐことになることは間違いない。

チームの仲間には、できる限りオープンで正直な態度で接しましょう。彼らの質問には直接答え、障壁になるのではなく、情報のパイプ役として行動しましょう。もし明かせないと思うことがあれば、そう言ってください。チームはあなたの誠実さに感謝し、情報を伝え、正直で正確な見積もりを作成することで応えてくれるでしょう。

平等 - 公平で公平であること

私がマネージャーとして生きてきた格言のひとつに、「個人は成功できるが、失敗できるのはチームだけだ」というものがある。基本的にこれは、人前では賞賛はどこまでも与えるべきだが、批判は決してしてはいけないということだ。人前で、仲間の前で批判されることは、誰にとってもモチベーションを高める力にはならない。

プロジェクトで取り組むべき問題があれば、通常、チーム全体で取り組むべき課題として提起することができる。問題に対する負担を共有することで、ほとんどのチームは問題解決のために協力し合う。問題を一人または複数の個人に押し付けると、チームが分裂し、対立が生じることが多い。問題をオープンに議論することで、チームは問題のオーナーシップを持ち、自分たちで解決するようになる。

忠誠心 - チームを守る

一方ではクライアントにプロジェクトの成功を約束する義務があり、もう一方ではチームを代表してお互いをサポートする責任がある。通常、この2つの目的はきちんと一致するはずだが、必ずしもそうではない!

どちらかを選ばなければならない状況では、道徳的に難しい立場を取る必要がある。クライアントに自分の汚点をぶちまけてはいけない。チームの仲間と状況を話し合い、解決策を考え、それをクライアントに提示する。

任せることを学ぶ

軍隊ではよく、将校が出すべき命令は "少佐に続け！ "だけだというジョークが飛び交う。将校は指揮を執ることが期待されており、実際に物事を成し遂げるのは、その任務により適した者、つまり部隊に任されている。もし仕事を分担するのであれば、適切な権限委譲を行うこと。

適切な権限委譲とは、誰かが仕事を理解できるように、合理的で達成可能な目標を設定し、仕事を成し遂げるために必要なサポートをすべて与えることである。

また、彼ら自身に仕事を終わらせるだけの余裕を与えることも必要だ。彼らに仕事を任せれば、彼らもあなたを放っておいて仕事に取りかかるだろう。彼らの肩越しに時間を費やしても、彼らを困らせ、あなたの貴重な時間を無駄にするだけだ。

ちょっと待て、まだ終わっていないぞ!

さて、プロジェクトの準備は整った。開発も終わり、テストも徹底的に行われた。しかし、あなたはまだそれを怒りにまかせて使おうとするクライアントにロールアウトしなければならない。

ローンチの前には、すべての準備が整い、プロジェクトがリリースに適した状態であることを確認する（比較的短い）期間があります。ローンチに先立ち、最終テストを行い、製品を使用する本番環境へのリリースを計画したい。この時点では、土壇場での不測の事態を避けるため、しっかりとした計画的なプロセスが重要です。

受け入れテスト

受け入れテストは、これまでのテスト作業とは別の重要な段階を形成する。機能テストや技術テストとは異なり、受け入れテストは純粋に製品の「受け入れ」に焦点を当てます。顧客はそれを望んでいるのか? これは商業プロジェクトにおける報酬であり、小切手にサインし、金銭を授受する場である。

正式なプロジェクトであれば、プロジェクトの早い段階、おそらく設計段階で受け入れテストを定義し、クライアントと署名しておく必要があるでしょう。

リリースコントロール

これは些細なことのように思えるかもしれないが、顧客が問題を発見したときに修正や変更を加えることができるように、物事を追跡しておく必要がある。

これらは通常、リリースごとに追跡する必要があるものです:

- リリースのバージョン番号または識別子
- リリースの日時
- リリースの目的（メンテナンス、バグフィックス、テストなど）
- リリース内の各コンポーネントについて
    - コンポーネントの名前とバージョン番号
    - 最終更新日

以下はその例である:

バージョン	日付	タイプ	コンポーネント			
			名称	バージョン	最後の改造	ハッシュ
3.0.5.28	27-05-2004	内部	カーネル	2.3.0.25	25-05-2004	#12AF2363
			GUI	1.0.0.12	27-05-2004	#3BC32355
			I/Oドライバ	2.3.0.01	01-04-2004	#B321FFA
3.0.5.29	05-06-2004	内部	カーネル	2.3.0.28	03-06-2004	#2C44C5A
			GUI	1.0.0.15	01-06-2004	#32464F4
			I/Oドライバ	2.3.0.01	01-04-2004	#B321FFA
3.0.5.30	その他...	...	...	...	...	...

ポジティブな認識

技術者の多くは、ビジネスの広報面が苦手だ。コミュニケーションやマーケティングは一般的に得意分野ではない。しかし、私たちがどんなに違う考え方をしたくても、人は認識に基づいて決断を下します。あなたのプロジェクトは、肯定的な結果を生み出すのと同じくらい、肯定的な*認識を*生み出す必要があります。

シャンパンを流し、マーチングバンドを演奏するようなブラックタイのイベントではないかもしれませんが、プロジェクトの立ち上げを象徴するようなイベントが必要です。少なくとも、関係者全員にメールなどで一般的なアナウンスをして、プロジェクトを立ち上げるべきです。

最後に、立ち上げには時間と組織が必要であり、適切な計画とスケジュールを立てることを忘れないでください（多くの不測の事態に備えて！）。発売日は必然的に固定され、多くの宣伝が行われるため、その日を逃したり、延期せざるを得なくなったりすると、非常に恥ずかしいことになります。プロジェクトの初期段階では、発売日を漠然と決めておき、最終テストを終えて自信が持てたら、具体的な発売日を発表することができます。

> 立ち上げのプロセスでは、チームの一部または全員を立ち会わせることをお忘れなく。これには2つの重要なメリットがあります。これには2つの重要な利点がある。まず第一に、彼らの努力に値する評価を与えることだ。確かに、あなたは自分の役割を果たしたが、あなたが最後にしたいことは、象牙の塔に座ってすべての手柄を主張することだ。第二に、あなたのPRキャンペーンに素晴らしい燃費をもたらすことができる。正直なところ、人々は経営陣と話すよりも、泥にまみれて苦労し、自分の眉間に汗してプロジェクトを成功させた個人と話したいものだ!

メンテナンスとアップグレード

プロジェクトに永続性があれば、メンテナンスやアップグレードが必要になります。ユーザーの要求をすべて満たすために最善を尽くしたつもりでも、それらの要求は時間とともに進化し、製品もそれに合わせて進化していかなければなりません。

保守性の重要な部分は、あなたのプロジェクト、設計文書、支援資料を、あなたの後に続く人たちの重要な参考資料として利用できるようにすることです。ユーザーの将来的なニーズに対応するために、彼らがシステム内の変更をうまく実装するには、あなたの考え方を理解する必要があります。

アップグレードのスケジューリング

アップグレードは常に必要です。アップグレードを無視することも、前もって計画することもできます。もし、アップグレードやメンテナンスリリースが必要であると認識しているのであれば、プロジェクトの最初の立ち上げから3ヶ月後にアップグレードを計画することができます。

これにより、プロジェクトチームのメンバーは、最初のローンチ作業から次のリリースの設計や仕様策定に移ることができるため、中断を防ぐことができます。彼らは、前のリリースから残っている不具合の「トリアージ」を開始し、どれをメンテナンスリリースで修正し、どれを修正しないかを決定することができます。

この考え方をさらに発展させると、（単発の製品リリースではなく）継続的なリリースを行う環境で作業している場合は、製品のポイントリリースを定期的にスケジュールすることを検討すべきです。例えば、1年ごとにメジャーリリースを行う製品を開発している場合、メンテナンスのために6ヶ月ごと、場合によっては3ヶ月ごとにマイナーリリースを行うことを検討すべきです。定期的なリリーススケジュールは、あなたのチーム、顧客、サポート担当者がリリースを予測し、リリースを計画するのに役立ちます。

サポート

あなたのプロジェクトは素晴らしいローンチを迎え、何百人もの熱心なユーザーを獲得しましたが、まだ終わっていません。製品の使い方や適切な設定をユーザーに説明するのは誰ですか? もし、ユーザーが製品のバグを見つけたら?

トレーニング

準備の整っていないサポート・チームに製品やシステムを丸投げしても、評価されることはありません。このトレーニングの規模は、プロジェクトの規模によって異なります。

最下層では、ユーザーマニュアルとサポートマニュアルの提供で十分でしょう。製品やシステムがそれほど複雑でなければ、それで十分かもしれません。より複雑な製品であれば、チュートリアルやシミュレーションなど、サポートチームがスピードアップできるような特別なトレーニングガイドやサポート資料が必要になるかもしれません。

また、プロジェクトチームのメンバーを使って、サポートアナリストを育成することも検討しよう。その製品に携わった人たちほど、その製品のことをよく知っている人はいないでしょうし、その人たちの何人かは、おそらく、そのシステムがロールアウトした後、サードレベルのサポートとして活動することになるでしょう。このような情報源をサポートチームに開放することで、彼らの評価とサポートを得ることができ、プロジェクトチームは、実際に製品を直接使用することになる人々との交流を楽しむことができるかもしれません。

コメント

プロジェクト・マネジメントは難しく、感謝されることのない仕事である。物事がうまくいかないとき、あなたは責任を負わなければならない。

しかし、プロジェクトには驚くべき報酬が伴う。賢明なビジネス・マネジャーは、ビジネスに真の価値をもたらすのは、最も賢い技術専門家でも、最も口がうまいセールスマンでもないことを知っている。物事を成し遂げることができる人材なのだ。一貫して成果を出し続ける人材は、すぐに問題解決の名声を獲得し、それに応じて報酬を得ることになる。

とはいえ、物事を成し遂げるのは必ずしも容易ではない。政治、リソースの制限、技術的な問題、個人的な相違、組織的な妨害など、すべてが成功の妨げになる可能性がある。

本書のアイデアは、新しい独自のモデルでもなければ、プロジェクトマネジメントに対する神秘的な洞察でもない。本書は、実際のプロジェクトに携わり、既存の方法論に対する純粋な不満から導き出されたものである。

本書のアイデアを使えば、どんなプロジェクトでも計画、管理、実行できるようになるはずだ。これは、チーム全体が理解しやすいモデルであり、運が良ければ、プロジェクトを時間通り、目標通り、予算内で遂行することができるだろう。

幸運を祈る

よ、ニック

・ジェンキン

ス。

受入テスト（Acceptance Test）製品の状態を評価し、エンドユーザーへの受け入れ態勢を判断するための最終的な機能テスト。

受入基準（Acceptance Criteria）製品またはシステムが受入試験（Acceptance Test）において判断される基準。

アルファ　製品の最初のバージョンで、意図された機能はすべて実装されているが、インターフェイスは完成しておらず、バグも修正されていない。

ベースライン　スケジュール・ベースライン」とは、プロジェクト計画の一部の特定の時点でのスナップショットのこと。時間の経過とともに比較することができる。

ベータ版　すべての機能が実装され、インターフェースも完成しているが、まだ問題や欠陥がある製品の最初のバージョン。

ビッグバン　新システムを「一挙に」導入すること。旧システムから新システムへの移行が（事実上）瞬時に行われる点で、インクリメンタルとは異なる。

ブラックボックステスト　製品の内部動作を知らずにテストを行うこと。製品の動作を確認するため、期待される結果と性能を比較する。

ボトム・アップ（Bottom Up）　初歩的な構成要素から製品を構築または設計し、小さな要素から始めて、より大きな構造へと発展させること。トップダウン」を参照。

クリティカルパス　あるフェーズやプロジェクトを完了させるために、最低限完了させなければならないタスクのセット。

納品物　マイルストーンで「納品」または完成させなければならない、具体的で物理的なもの。この用語は、煙や騒音の中で手触りのある最終製品を意味するために使われる。

エンドユーザー　あなたの製品が完成したとき、それを手にする哀れな人たち！あなたの製品が開発され、実装された後、それを実際に使用する人々。

Feature/Scope　creep（フィーチャークリープ／スコープクリープ）プロジェクトが自己増殖し、当初意図した以上のフィーチャや機能を取り込んでしまう容赦ない傾向。

インクリメンタル開発　製品を断片的に開発することで、全体を完成させることなく、徐々に機能を実装していくこと。

マイルストーン　プロジェクトのスケジュールの中で、プロジェクトの重要な部分の納品を示す重要なポイント。通常、特定の「成果物」に関連する。

プロトタイプ　プロジェクトにおいて設計上の決定を下すために使用される製品の単純なモデル。

品質保証（QA）　ベストプラクティスを通じて、製品に欠陥が入るのを防ぐプロセス。すでに存在する欠陥を取り除くために行われるテストと混同しないように。

要件　提供されるシステムまたは製品に要求される属性を特定する、プロジェクト利害関係者からのニーズの声明。

投資利益率　「ROI（Return On Investment）とは、プロジェクトの金銭的支出と金銭的利益とを比較する比率。プロジェクトの成功を示すために使われる。

ショー・ストッパー（Show Stopper）文字通りすべてを止めてしまうほど深刻な欠陥。通常、解決するまで優先的に注意を払う。

ステークホルダー　プロジェクトとその設計の成功に既得権益を持つ、顧客企業またはエンドユーザーの代表者。

テスト　製品の欠陥を発見し、リリース準備の現状を判断するために、製品を批判的に評価するプロセス。

Top Down　上位の構造を構築し、その構造のギャップを埋めることによって製品を構築または設計すること。対比については「ボトムアップ」を参照

ユーザビリティ　使いやすく、理解しやすく、操作しやすい製品の本質的な品質。しばしば、ユーザーを*困らせない製品の能力と*表現される。

ホワイト・ボックス／グラス・ボックス・テスト 製品がどのように機能するかを理解した上でテストすること。ブラックボックス・テストを参照。